UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Escola Superior d'Enginyeries Industrial,
Aeroespacial i Audiovisual de Terrassa

Project - Lambert Problem

Leoni Fritsch, Martí Güell, Gerard Martínez

Advanced Astrodynamics

ABSTRACT

This project develops a compact Python mission-analysis toolchain for preliminary interplanetary transfer design under a heliocentric two-body (patched-conic) approximation. The workflow combines (i) fast analytical planetary ephemerides, (ii) a robust Lambert boundary-value solver based on the Simó auxiliary-variable formulation, and (iii) automated launch-window exploration via pork-chop plots (PCPs), complemented by fixed-date minimum-ΔV targeting and transfer-orbit element extraction. Planetary states are generated from Standish–Williams mean orbital elements with linear secular propagation; Kepler's equation is solved using Halley iteration, and states are expressed in a heliocentric ecliptic J2000 frame with canonical AU/day units. The ephemeris model is validated against JPL Horizons, yielding position discrepancies on the order of 10^{-5} AU for Earth and 10^{-2} AU for Saturn at the demonstrated test epoch, consistent with the limitations of mean-element models. The Lambert solver employs split-region bracketing and bisection to handle elliptic and hyperbolic regimes, achieving time-of-flight residuals near 10^{-12} in canonical validation cases and matching a reference GUI, including a multi-revolution extension that reproduces dual-branch solutions. The PCP module scans departure date and time-of-flight grids to identify low-cost regions using the screening metric $\Delta V_T = \|\Delta v_1\| + \|\Delta v_2\|$; representative minima include an Earth–Mars prograde window with $\Delta V_T \approx 6.32$ km/s and inner/outer-planet trends consistent with increasing phasing sensitivity and transfer durations. Overall, the toolchain provides a transparent and computationally efficient baseline for launch-window surveys and transfer characterization in early mission design.

CONTENTS

1 Analytical Planetary Ephemerides for Pork Chop Plot Generation **4**

1.1 Introduction 4

1.2 Methodology 4

Reference Frame, Units, and Time Systems • Mean Orbital Elements and Linear Secular Propagation • From Mean Elements to Anomalies • State Vectors in the Perifocal Frame • Rotation to the Heliocentric Ecliptic J2000 Frame • High-Level Ephemeris Interface and Unit Conversion

1.3 Results 7

Numerical Validation against JPL Horizons

2 Lambert Boundary-Value Problem Solver Using the Simó Formulation **9**

2.1 Introduction 9

2.2 Methodology 9

Software Structure and Step-by-Step Algorithm • Geometric Preprocessing: Directed Transfer Angle and Simó Constants P and Q • Stumpff-Like Functions Used by the Simó Formulation • Simó Transfer-Time Equation $\Delta t(z)$ • Numerical Solution for z: Split-Region Bracketing + Bisection • Recovery of Conic Parameters a and p • Terminal Velocity Computation from Orbit Geometry

2.3 Results 11

Canonical Validation Results and GUI Cross-Check • Multi-Revolution Extension and Validation

3 Pork-Chop Plot Tool for Launch Window Search **18**

3.1 Introduction 18

3.2 Methodology 18

Software Entry Point and Function Interface • Departure Window Determination (Manual vs. Synodic-Based) • Grid Generation and Data Structures • Trajectory Scan Loop and Lambert Calls • Delta-V Metric and Optional Masking • Tracking the Minimum Solution • Visualization: 2D PCP Heatmap and Contours • Visualization: 3D Geometry and Transfer Orbit Reconstruction

3.3 Results 21

Mars: comparison between CCW and CW transfers • Mercury: narrow launch opportunities and high-energy inner transfer • Venus: low-energy inner transfer with broad feasible region • Jupiter: multi-year transfers and synodic repetition • Saturn: longer TOF and increasing sensitivity to phasing • Uranus, Neptune, and Pluto: decades-to-centuries direct transfers

4 Minimum ΔV and Transfer-Orbit Elements for a Fixed Launch Date **28**

4.1 Introduction 28

4.2 Methodology 28

Software Inputs, Units, and Conventions • Lambert-Based ΔV_T Evaluation for a Fixed Launch Date • Two-Stage Search Strategy (Coarse + Local Refinement) • Conversion from Cartesian State to Keplerian Elements • Program Flow and Generated Output

4.3 Results 30

Interpretation of the Inner-Planet Transfers • Outer-Planet Transfers and Scaling Trends • Orbital-Element Patterns • Summary

5 Computation Time **32**

6 Conclusion **33**

INTRODUCTION

Interplanetary mission design problems can be framed as a trade-off between *when* to depart, *how long* to fly, and *how much* velocity change (ΔV) is required to achieve the transfer. In early design phases, this trade space is typically explored with simplified dynamical models that are fast, smooth over large date grids, and transparent to interpret. The goal of this project is to develop a compact Python-based mission analysis toolchain that supports launch-window exploration and transfer characterization using a heliocentric two-body (patched-conic) approximation.

The workflow implemented in this report is organized as a sequence of building blocks, each providing a capability needed by the next:

- **Analytical planetary ephemerides:** a lightweight routine that returns heliocentric planet state vectors $(\mathbf{r}, \mathbf{v})$ for arbitrary epochs, enabling rapid evaluation of departure and arrival boundary conditions over large time grids.
- **Lambert boundary-value solver:** a solver that computes the heliocentric transfer velocities that connect two position vectors in a prescribed time of flight, including robust handling of different geometric branches and extreme time-of-flight cases.
- **Pork-chop plot (PCP) generation:** a systematic scan over departure date and time of flight to map the transfer cost and identify low-ΔV launch windows, complemented by a 3D visualization of the corresponding transfer geometry.
- **Fixed launch-date targeting:** for a given departure date, a targeted search for the minimum-ΔV transfer to each planet and the extraction of the associated transfer-orbit elements.

Throughout the report, transfers are evaluated in a Sun-centered heliocentric frame using a consistent unit convention (AU and day internally, with ΔV reported in km/s). The selected cost metric is the sum of the magnitudes of the departure and arrival heliocentric velocity mismatches, which serves as an effective screening proxy for preliminary launch-window surveys.

The remainder of this document is organized as follows. Section 1 presents the analytical ephemerides model and validates it against a high-precision reference. Section 2 describes the Lambert solver implementation and verifies it using canonical test cases and a reference GUI. Section 3 applies the solver within a pork-chop plot tool to identify and visualize launch windows for multiple target planets. Section 4 reports the minimum-ΔV solutions and the corresponding transfer-orbit elements for a fixed launch date. Section 5 summarizes the computational time required to generate the planetary-transfer pork-chop plots. Finally, Section 6 provides the concluding remarks.

1 ANALYTICAL PLANETARY EPHEMERIDES FOR PORK CHOP PLOT GENERATION

1.1 Introduction

The first building block of the interplanetary mission design tool is a compact analytical ephemeris routine that returns heliocentric position and velocity vectors of the planets for arbitrary departure and arrival dates. The implementation follows the classical workflow "mean elements → anomalies → state vectors" using the Standish–Williams mean orbital elements with linear secular rates [1]. While this approach is less accurate than modern numerical ephemerides (e.g. JPL DE series), it is well suited for preliminary mission design studies such as pork chop plots, where consistency, speed, and smooth trends across large date grids are essential.

1.2 Methodology

1.2.1 Reference Frame, Units, and Time Systems

Reference frame. All states are expressed in a heliocentric inertial frame aligned with the mean ecliptic and mean equinox of J2000.0, consistent with the element definitions in Standish & Williams [1]. In the code, the final transformation from the perifocal frame uses the standard 3–1–3 Euler sequence $R_3(\Omega)R_1(i)R_3(\omega)$ as commonly presented in astrodynamics references [2], [3].

Canonical astronomical units. The computations are carried out primarily in the canonical unit system (AU, day). The Gaussian gravitational constant is set to the exact value

$$k = 0.01720209895 \text{ rad/day}, \tag{1}$$

and the Sun's heliocentric gravitational parameter is represented in canonical units as

$$\mu_\odot = k^2 \quad [\text{AU}^3/\text{day}^2], \tag{2}$$

which is a standard convention in analytical solar-system modeling [1], [4]. For optional conversion to SI-like units, the implementation uses the exact IAU 2012 definition of the astronomical unit [5] and the exact day length:

$$1 \text{ AU} = 149\,597\,870.7 \text{ km}, \qquad 1 \text{ day} = 86400 \text{ s}, \tag{3}$$

so that

$$1 \text{ AU/day} = \frac{149\,597\,870.7 \text{ km}}{86400 \text{ s}} \approx 1.73146 \text{ km/s}. \tag{4}$$

Time representation (JD and JC). Input epochs are specified as UTC calendar dates at 00:00. The code converts the date to a Julian Date JD and then to Julian centuries since J2000.0:

$$T = \text{JC} = \frac{\text{JD} - 2451545.0}{36525}. \tag{5}$$

The implemented `JD_from_UTC` routine applies the standard January/February shift and a compact integer-part formulation for 0h UTC (see, e.g., the general discussion of Julian-date algorithms in [3]). This T is the independent variable for the secular propagation of the mean elements.

1.2.2 Mean Orbital Elements and Linear Secular Propagation

Standish–Williams element set. For each planet, the code stores the mean elements at J2000.0 and their secular rates per Julian century (Table 8.10.2 in [1]):

- semi-major axis a [AU],
- eccentricity e [–],

- inclination i [deg],
- mean longitude L [deg],
- longitude of perihelion ϖ (denoted `w_bar` in the code) [deg],
- longitude of the ascending node Ω (denoted `omega` in the code) [deg].

These values are implemented as dictionaries `X_0` (epoch values) and `X_punkt` (rates), grouped per planet in `PLANETS`.

Linear secular propagation. At a given epoch T from Eq. (5), each element is propagated with the linear model

$$x(T) = x_0 + \dot{x}T, \tag{6}$$

implemented as `propagation_equation(x_0, x_dot, jc)`. Angular quantities are wrapped to a standard range using `deg_correction`:

$$\alpha \leftarrow \alpha \bmod 360^\circ. \tag{7}$$

This wrapping prevents discontinuities in subsequent anomaly computations and ensures numerical stability when scanning large date grids for pork chop plots.

Model variants for pork chop plot fidelity. To support different levels of idealization, the routine provides three element models:

- `full`: propagate a, e, i with Eq. (6) as given in [1];
- `planar_elliptic`: propagate a, e but set $i \equiv 0$ to enforce coplanar motion;
- `circular_coplanar`: propagate a only, set $e \equiv 0$ and $i \equiv 0$ for a fully idealized planar model.

These controlled simplifications are useful to isolate the impact of eccentricity and inclination on transfer opportunities in preliminary analyses.

1.2.3 From Mean Elements to Anomalies

Derived angles. From the propagated mean elements, the argument of perihelion ω and the mean anomaly M are computed as

$$\omega = \varpi - \Omega, \qquad M = L - \varpi, \tag{8}$$

consistent with standard orbital-element definitions [1], [2], [3]. In the code, these operations are performed by `arg_perihelion_deg` and `mean_anomaly_deg` and subsequently wrapped to $[0^\circ, 360^\circ)$.

Kepler equation (elliptic) and Halley iteration. For $0 \le e < 1$, the eccentric anomaly E is obtained from Kepler's equation

$$M = E - e\sin E. \tag{9}$$

Rather than Newton's method, the implementation uses Halley's method via `halley_solver(M_deg,e)`. Defining

$$f(E) = E - e\sin E - M, \qquad f'(E) = 1 - e\cos E, \qquad f''(E) = e\sin E, \tag{10}$$

the Halley update is

$$E_{k+1} = E_k - \frac{f(E_k)}{f'(E_k) - \frac{1}{2} f(E_k)\dfrac{f''(E_k)}{f'(E_k)}}. \tag{11}$$

This incorporates curvature information through $f''(E)$ and typically yields a higher local convergence order than Newton-type updates for smooth nonlinear equations [2], [3]. Several stability measures present in the code are important for robust batch evaluations across many dates:

- M is converted from degrees to radians and normalized to $[0, 2\pi)$ (implemented using `TWOPI` and the modulus operation), preventing slow convergence for large unwrapped anomalies.
- The initial guess is chosen as $E_0 = M$ for moderate eccentricity and $E_0 = \pi$ for larger e (threshold $e \approx 0.8$ in the code), a common practical strategy to avoid poor initializations at high e [2], [3].
- The effective denominator in Eq. (11) is guarded against extremely small values (via a floor 10^{-14}) to prevent unstable steps when $f'(E)$ becomes small.

Iteration stops when $|\Delta E| < 10^{-12}$ rad or after a maximum number of iterations, and the solver additionally returns an iterate history for diagnostics (`hist`).

True anomaly conversion. After convergence, the true anomaly θ is computed from E using the standard trigonometric relations

$$\sin\theta = \frac{\sqrt{1-e^2}\,\sin E}{1-e\cos E}, \tag{12}$$

$$\cos\theta = \frac{\cos E - e}{1-e\cos E}, \tag{13}$$

$$\theta = \operatorname{atan2}(\sin\theta, \cos\theta), \tag{14}$$

and wrapped to $[0, 2\pi)$ using `rad_correction` [2], [3]. This step is implemented in `true_anomaly(E,e)`.

1.2.4 State Vectors in the Perifocal Frame

Specific angular momentum. Using canonical units, the specific angular momentum magnitude is computed as

$$h = \sqrt{\mu_\odot a(1-e^2)} \quad [\mathrm{AU}^2/\mathrm{day}], \tag{15}$$

implemented in `angular_momentum(a,e)`. Equation (15) follows from two-body Keplerian dynamics in the elliptic case [2], [3].

Perifocal position and velocity. In the perifocal frame (PQW), the radial distance is evaluated as

$$r = \frac{h^2/\mu_\odot}{1+e\cos\theta}, \tag{16}$$

and the corresponding state vectors are

$$\mathbf{r}_{\mathrm{PF}} = \begin{bmatrix} r\cos\theta \\ r\sin\theta \\ 0 \end{bmatrix}, \qquad \mathbf{v}_{\mathrm{PF}} = \frac{\mu_\odot}{h}\begin{bmatrix} -\sin\theta \\ e+\cos\theta \\ 0 \end{bmatrix}, \tag{17}$$

as implemented by `r_v_pf(h,e,theta)` [2], [3].

1.2.5 Rotation to the Heliocentric Ecliptic J2000 Frame

The perifocal vectors are rotated into the heliocentric ecliptic J2000 frame via

$$\mathbf{r}_{\mathrm{ecl}} = \mathbf{Q}\mathbf{r}_{\mathrm{PF}}, \qquad \mathbf{v}_{\mathrm{ecl}} = \mathbf{Q}\mathbf{v}_{\mathrm{PF}}, \tag{18}$$

with

$$\mathbf{Q} = R_3(\Omega)\,R_1(i)\,R_3(\omega), \tag{19}$$

where R_1 and R_3 are the elementary rotation matrices about the x- and z-axes, respectively [2]. In the implementation `pf_to_ecliptic(elem, r_pf, v_pf)`, the input angles are provided in degrees and converted to radians using `DEG2RAD`. The argument of perihelion is reconstructed consistently as $\omega = \varpi - \Omega$ from the stored element set.

1.2.6 High-Level Ephemeris Interface and Unit Conversion

All steps are wrapped into a single high-level function `planet_state(planet_name, year, month, day, mode`

1. Compute JD and $T =$ JC from the input date.
2. Propagate $(a, e, i, L, \varpi, \Omega)$ with Eq. (6) and apply the selected model (`full`, `planar_elliptic`, `circular_coplanar`).
3. Compute M and ω, solve Eq. (9) using Halley's iteration (Eq. (11)), and convert to θ.
4. Construct $(\mathbf{r}_{\mathrm{PF}}, \mathbf{v}_{\mathrm{PF}})$ and rotate to $(\mathbf{r}_{\mathrm{ecl}}, \mathbf{v}_{\mathrm{ecl}})$.
5. Optionally convert AU $\to$ km and AU/day $\to$ km/s via the exact constants from [5].

This interface produces planetary ephemerides suitable for scanning departure and arrival date grids and feeding the Lambert solver in later stages of the project.

1.3 Results

1.3.1 Numerical Validation against JPL Horizons

To verify the correctness of the analytical ephemerides, the computed planetary states are compared against high-precision reference ephemerides from the JPL Horizons online system [6]. The comparison is performed in the same reference setting used by the solver: heliocentric location (`@sun`), ecliptic reference plane (`refplane="ecliptic"`), mean equinox of J2000, and units AU and AU/day. Reference vectors are obtained programmatically via the `astroquery` interface [7], using NAIF IDs (e.g. 399 for Earth and 699 for Saturn).

For a given UTC date at 00:00, the model produces $(\mathbf{r}_{\mathrm{model}}, \mathbf{v}_{\mathrm{model}})$ through the pipeline described above (secular mean elements [1], Halley solve of Kepler's equation, PQW state, and 3–1–3 rotation [2], [3]). Horizons provides the reference state $(\mathbf{r}_H, \mathbf{v}_H)$ at the same Julian Date. The difference vectors and relative errors are defined as

$$\Delta\mathbf{r} = \mathbf{r}_{\mathrm{model}} - \mathbf{r}_H, \qquad \Delta\mathbf{v} = \mathbf{v}_{\mathrm{model}} - \mathbf{v}_H, \tag{20}$$

$$\varepsilon_r^{\mathrm{rel}} = \frac{\|\Delta\mathbf{r}\|}{\|\mathbf{r}_H\|}, \qquad \varepsilon_v^{\mathrm{rel}} = \frac{\|\Delta\mathbf{v}\|}{\|\mathbf{v}_H\|}. \tag{21}$$

These measures are computed in the function `compare_with_horizons(...)`.

Example case (Earth and Saturn, 2017–09–15). As a representative demonstration, Earth and Saturn are evaluated at 2017–09–15 00:00 UTC, corresponding to JD = 2458011.5. The analytical model returns the following heliocentric ecliptic J2000 states (AU, AU/day):

$$\mathbf{r}_{\oplus,\mathrm{model}} = \begin{bmatrix} 9.96191705 \times 10^{-1} \\ -1.38063392 \times 10^{-1} \\ 5.56012507 \times 10^{-6} \end{bmatrix} \text{AU}, \qquad \mathbf{v}_{\oplus,\mathrm{model}} = \begin{bmatrix} 2.08179888 \times 10^{-3} \\ 1.69769631 \times 10^{-2} \\ -6.83700704 \times 10^{-7} \end{bmatrix} \text{AU/day}, \tag{22}$$

$$\mathbf{r}_{\mathrm{Sat,model}} = \begin{bmatrix} -5.2093175 \times 10^{-1} \\ -1.003118403 \times 10^{1} \\ 1.9518666 \times 10^{-1} \end{bmatrix} \text{AU}, \qquad \mathbf{v}_{\mathrm{Sat,model}} = \begin{bmatrix} 5.26714 \times 10^{-3} \\ -3.0640 \times 10^{-4} \\ -2.0423 \times 10^{-4} \end{bmatrix} \text{AU/day}. \tag{23}$$

The corresponding Horizons comparison (printed by the code) yields the error norms and relative errors summarized in Table 1. For Earth, the position discrepancy is on the order of 10^{-5} AU, while for Saturn it increases to

10^{-2} AU, consistent with the known limitations of a mean-element secular model compared to full numerical ephemerides [1], [6]. The velocity discrepancies remain on the order of 10^{-6} AU/day for both bodies.

Table 1. Example validation against JPL Horizons (2017–09–15 00:00 UTC, JD = 2458011.5), heliocentric ecliptic J2000, units AU and AU/day. Values taken directly from the program output.

Planet	$\|\Delta\mathbf{r}\|$ [AU]	$\|\Delta\mathbf{v}\|$ [AU/day]	$\varepsilon_r^{\mathrm{rel}}$	$\varepsilon_v^{\mathrm{rel}}$
Earth	3.829×10^{-5}	7.625×10^{-6}	3.808×10^{-5}	4.457×10^{-4}
Saturn	1.551×10^{-2}	7.467×10^{-6}	1.541×10^{-3}	1.413×10^{-3}

Reproducibility (raw console output). For traceability, the exact diagnostic printout for this run is reproduced below in abbreviated form:

```
Earth r [AU]   = [ 9.96191705e-01 -1.38063392e-01  5.56012507e-06]
Earth v [AU/d] = [ 2.08179888e-03  1.69769631e-02 -6.83700704e-07]
Saturn r [AU]  = [ -0.52093175 -10.03118403   0.19518666]
Saturn v [AU/d]= [ 0.00526714 -0.0003064  -0.00020423]

=== Earth { Comparison with JPL Horizons (JD = 2458011.5) ===
||r|| = 3.8291835e-05 AU ; ||v|| = 7.6248501e-06 AU/day
Relative errors:  ||r||/||r_H|| = 3.808e-05 ; ||v||/||v_H|| = 4.457e-04

=== Saturn { Comparison with JPL Horizons (JD = 2458011.5) ===
||r|| = 1.5505377e-02 AU ; ||v|| = 7.4671821e-06 AU/day
Relative errors:  ||r||/||r_H|| = 1.541e-03 ; ||v||/||v_H|| = 1.413e-03
```

Overall, this validation confirms that the implemented `planet_state` function is internally consistent with the adopted analytical mean-element model [1] and reproduces the expected order of accuracy relative to modern high-precision ephemerides [6], [7] for preliminary interplanetary transfer design and pork chop plot generation.

2 LAMBERT BOUNDARY-VALUE PROBLEM SOLVER USING THE SIMÓ FORMULATION

2.1 Introduction

The mission design tool requires a Lambert solver to compute the transfer velocities that connect two heliocentric position vectors $\mathbf{r}_1$ and $\mathbf{r}_2$ in a prescribed time of flight Δt. In the two-body model, the Lambert problem is posed as:

$$\mathbf{r}(t_1) = \mathbf{r}_1, \qquad \mathbf{r}(t_2) = \mathbf{r}_2, \qquad t_2 - t_1 = \Delta t, \tag{24}$$

with gravitational parameter μ.

In a first approach, the single-revolution solution is computed through the *Simó auxiliary-variable formulation* as presented in the open literature (e.g., De La Torre et al. and related Lambert reviews) [8], [9], [10]. The method introduces a scalar variable z and a transfer-time map $\Delta t(z)$. Since $\Delta t(z)$ is not inverted analytically, z is obtained numerically. Then the conic parameters (a, p) are reconstructed and terminal velocities $(\mathbf{v}_1, \mathbf{v}_2)$ are computed from two-body polar relations.

2.2 Methodology

2.2.1 Software Structure and Step-by-Step Algorithm

The solver follows the same functional structure as the Python implementation:

1. **Vector normalization:** `unit(v)` returns $\hat{\mathbf{v}}$ and raises an error for near-zero norms.
2. **Stumpff-like functions:** `stumpff_c0c1c2c3(z)` evaluates (c_0, c_1, c_2, c_3) for elliptic ($z > 0$), hyperbolic ($z < 0$), and near-parabolic ($z \approx 0$) regimes.
3. **Geometry preprocessing:** `lambert_geometry(r1,r2,tm)` computes $(r_1, r_2, \Delta\theta, P, Q)$ using a *directed* transfer angle consistent with the requested direction flag `tm`.
4. **Transfer-time map:** `simo_dt(z,P,Q,mu)` evaluates $\Delta t(z)$ with domain checks (non-finite values return NaN).
5. **Solve for z:** `solve_z(dt_target,...)` uses a robust *split-region bracketing + bisection* strategy (elliptic branch $z > 0$ vs. hyperbolic branch $z < 0$) to satisfy $\Delta t(z) = \Delta t_{\text{target}}$.
6. **Recover (a, p):** `lambert_simo(...)` reconstructs $a(z)$ and $p(z)$.
7. **Terminal velocities:** `terminal_velocities_from_ap(...)` computes $(\mathbf{v}_1, \mathbf{v}_2)$ in the transfer plane (with a dedicated circular branch and collinearity handling).
8. **Validation:** canonical test cases are executed and compared against a reference GUI implementation [8].

Implementation note (own code). All core astrodynamics routines used in this project (analytical ephemerides, Kepler propagation, Lambert solver, and ΔV evaluation) were implemented by the authors. External packages are used only for basic numerical utilities (e.g., array handling) and plotting. When third-party tools are referenced (e.g., JPL Horizons via a query interface), they are used *exclusively* to retrieve reference data for validation and are not part of the computational pipeline.

2.2.2 Geometric Preprocessing: Directed Transfer Angle and Simó Constants P and Q

Given endpoint vectors $\mathbf{r}_1$ and $\mathbf{r}_2$, their magnitudes are $r_1 = \|\mathbf{r}_1\|$ and $r_2 = \|\mathbf{r}_2\|$. The Simó constants used by the implementation are

$$P = r_1 + r_2, \qquad Q = 2\sqrt{r_1 r_2}\cos\left(\frac{\Delta\theta}{2}\right). \tag{25}$$

Directed transfer angle consistent with `tm`. Instead of using an unsigned $\arccos(\cdot)$ angle, the code computes the *directed* angle in $[0, 2\pi)$ around a plane normal $\hat{\mathbf{n}}$:

$$\Delta\theta = \operatorname{atan2}\left(\frac{(\mathbf{r}_1 \times \mathbf{r}_2)\cdot\hat{\mathbf{n}}}{r_1 r_2}, \frac{\mathbf{r}_1 \cdot \mathbf{r}_2}{r_1 r_2}\right) \bmod 2\pi. \tag{26}$$

The normal $\hat{\mathbf{n}}$ is built from $\mathbf{r}_1 \times \mathbf{r}_2$ when non-collinear; otherwise a default $\pm\hat{\mathbf{z}}$ (or a provided plane normal) is used. The sign of $\hat{\mathbf{n}}$ is selected to match the requested direction: `tm=+1` (prograde / CCW) versus `tm=-1` (retrograde / CW).

2.2.3 Stumpff-Like Functions Used by the Simó Formulation

The implementation uses:

$$c_0(z) = \begin{cases} \cos(\sqrt{z}), & z > 0, \\ 1, & z = 0, \\ \cosh(\sqrt{-z}), & z < 0, \end{cases} \qquad c_1(z) = \begin{cases} \dfrac{\sin(\sqrt{z})}{\sqrt{z}}, & z > 0, \\ 1, & z = 0, \\ \dfrac{\sinh(\sqrt{-z})}{\sqrt{-z}}, & z < 0, \end{cases} \tag{27}$$

$$c_2(z) = \frac{1 - c_0(z)}{z}, \qquad c_3(z) = \frac{1 - c_1(z)}{z}. \tag{28}$$

For $|z|$ below a small threshold, the Taylor limits $c_0(0) = 1,\ c_1(0) = 1,\ c_2(0) = 1/2,\ c_3(0) = 1/6$ are enforced to avoid cancellation and maintain continuity [9], [10].

2.2.4 Simó Transfer-Time Equation $\Delta t(z)$

Let $c_0 = c_0(z)$ and $c_1 = c_1(z)$, and evaluate $c_{2,4} = c_2(4z)$ and $c_{3,4} = c_3(4z)$. The code implements:

$$\Delta t(z) = \frac{2P\,c_{3,4} + Q\,(c_1\,c_{2,4} - 2c_0\,c_{3,4})}{c_1^3}\sqrt{\frac{2\,(P - Qc_0)}{\mu}}. \tag{29}$$

Two domain checks are enforced: (i) c_1^3 must be non-zero, and (ii) the square-root argument must be non-negative. If violated, the function returns NaN, allowing the root-solver to stay in a valid region.

2.2.5 Numerical Solution for z: Split-Region Bracketing + Bisection

The root problem is $F(z) = \Delta t(z) - \Delta t_{\text{target}} = 0$. To robustly handle both elliptic and hyperbolic solutions, the implementation uses the parabolic transfer time at $z = 0$:

$$\Delta t_{\text{par}} = \Delta t(0). \tag{30}$$

If $\Delta t_{\text{target}} > \Delta t_{\text{par}}$, the solver brackets an **elliptic** solution on

$$z \in [0,\ \pi^2), \tag{31}$$

where π^2 is a practical upper bound for single-revolution elliptic cases in this formulation (as also used in reference implementations [8]).

If $\Delta t_{\text{target}} \le \Delta t_{\text{par}}$, the solver brackets a **hyperbolic** solution on $z < 0$ with

$$z \in [z_{\min},\ 0], \tag{32}$$

using $z_{\min} = -(\operatorname{arccosh}(P/Q))^2$ when $Q > 0$, and otherwise falling back to a safe large negative bound (expanded if needed) to avoid overflow in $\cosh(\cdot)$.

On each bisection iteration, the midpoint $z = (z_{\text{low}} + z_{\text{high}})/2$ is tested. If $\Delta t(z)$ is non-finite or exceeds the target, the upper bound is moved; otherwise the lower bound is moved. Iteration stops when the bracket width drops below `tol`. With `tol=1e-12`, the observed time residuals are typically $\mathcal{O}(10^{-12})$.

2.2.6 Recovery of Conic Parameters a and p

After solving for z, the conic parameters are reconstructed as:

$$a(z) = \frac{P - Qc_0(z)}{2z\,c_1(z)^2}, \tag{33}$$

and

$$p(z) = \frac{2A^2C^2}{P - Qc_0(z)}, \qquad A = \sqrt{r_1}, \qquad C = \sqrt{r_2}\sin\left(\frac{\Delta\theta}{2}\right). \tag{34}$$

Feasibility requires $P - Qc_0(z) > 0$; this is explicitly checked in the code.

2.2.7 Terminal Velocity Computation from Orbit Geometry

An in-plane basis is constructed from $\hat{\mathbf{n}}$ (transfer-plane normal), $\hat{\mathbf{r}}_i = \mathbf{r}_i/\|\mathbf{r}_i\|$, and $\hat{\mathbf{t}}_i = \frac{\hat{\mathbf{n}}\times\hat{\mathbf{r}}_i}{\|\hat{\mathbf{n}}\times\hat{\mathbf{r}}_i\|}$. The eccentricity is computed from $p = a(1-e^2)$ as

$$e = \sqrt{\left|1 - \frac{p}{a}\right|}, \tag{35}$$

which correctly yields $e \approx 0$ for circular cases and $e \gg 1$ for hyperbolic-like (rectilinear) cases produced by very short times of flight.

Circular special case. If e is below a small threshold, the code returns purely tangential velocities:

$$\mathbf{v}_i = \sqrt{\frac{\mu}{r_i}}\,\hat{\mathbf{t}}_i, \qquad v_{r,i} = 0. \tag{36}$$

General case. For $e > 0$, true anomalies are recovered from $r = p/(1 + e\cos\theta)$:

$$\cos\theta_i = \frac{p/r_i - 1}{e}, \tag{37}$$

and the branch ambiguity in θ_1 is resolved by choosing the candidate that best matches the second endpoint under $\theta_2 = \theta_1 + \Delta\theta$. Then polar components

$$v_r = \sqrt{\frac{\mu}{p}}\,e\sin\theta, \qquad v_t = \sqrt{\frac{\mu}{p}}\,(1 + e\cos\theta), \tag{38}$$

are mapped back to Cartesian form:

$$\mathbf{v}_i = v_{r,i}\hat{\mathbf{r}}_i + v_{t,i}\hat{\mathbf{t}}_i, \qquad i \in \{1,2\}. \tag{39}$$

A radial consistency check recomputes r_i from (p, e, θ_i) and reports `r_check_err`; the final time residual is reported as `dt_err`.

2.3 Results

2.3.1 Canonical Validation Results and GUI Cross-Check

The solver was validated in canonical units ($\mu = 1$) using five elementary test cases plus a Hohmann transfer (collinear endpoints). For each case, the printed output from the Python code is compared to the reference GUI values (GUI labels: *var*$\equiv z$, *sma*$\equiv a$, *slr*$\equiv p$, *ecc*$\equiv e$) [8].

GUI reference figures. Figures 1–6 show the GUI solutions for the same test settings used in the Python script.

Case 1: 90° **short-way (circular).** For $\mathbf{r}_1 = [1,0,0]$, $\mathbf{r}_2 = [0,1,0]$, $\Delta t = \pi/2 \approx 1.57$, the Python solver returns

$$\Delta\theta = 90^\circ, \quad z = 0.6168502751, \quad a \approx 1.0, \quad p \approx 1.0, \tag{40}$$

with `dt_err`$\approx 2.94 \times 10^{-13}$ and `r_check_err`=0. The velocities match the unit circular solution: $\mathbf{v}_1 = [0,1,0]$, $\mathbf{v}_2 = [-1,0,0]$. This is consistent with the GUI readout in Fig. 1.

Case 2: 270° **long-way (same circular orbit).** With the long-way time $\Delta t = 3\pi/2 \approx 4.71$ on the same endpoints, the solver returns

$$\Delta\theta = 270^\circ, \quad z = 5.5516524756, \quad a \approx 1.0, \quad p \approx 1.0, \tag{41}$$

with `dt_err`$\approx 6.53 \times 10^{-13}$ and `r_check_err`=0. The GUI reports $z \approx 5.55$ and the same (a, e, p) values of the unit circle (Fig. 2).

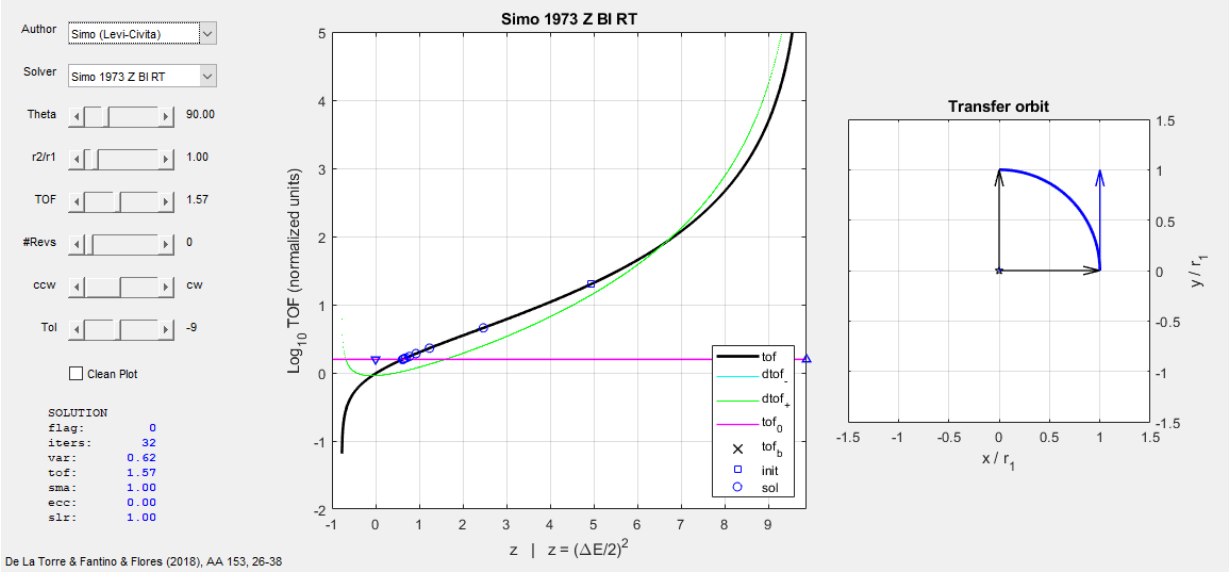


Figure 1. GUI reference – Case 1: 90° short-way, TOF ≈ 1.57. The GUI reports $z \approx 0.62$, $a \approx 1.00$, $e \approx 0$, $p \approx 1.00$.

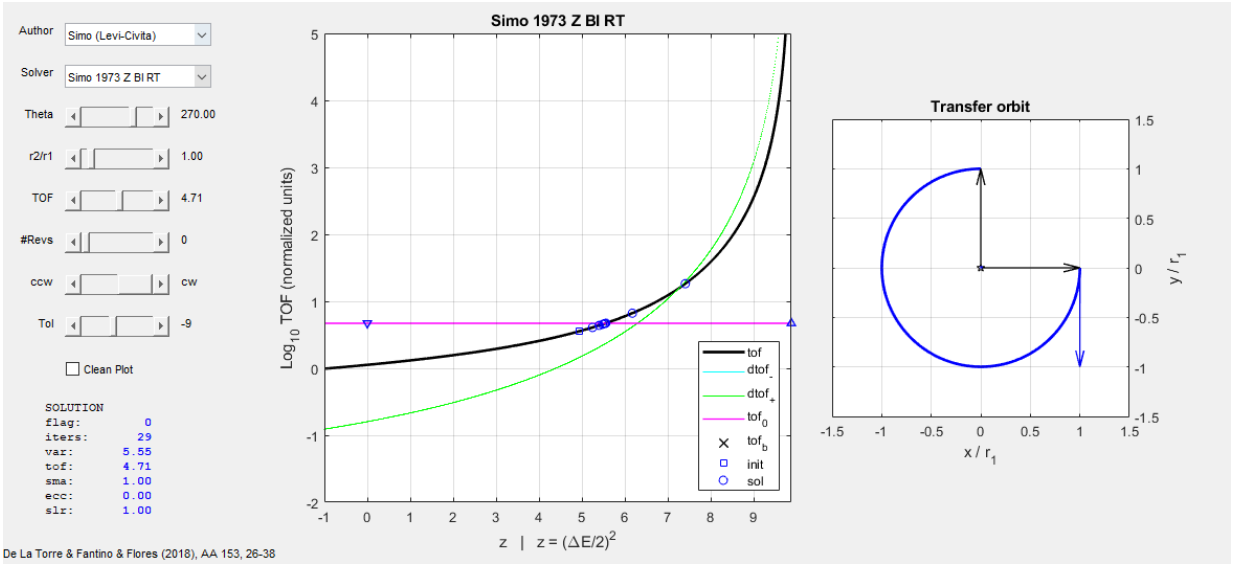


Figure 2. GUI reference – Case 2: 270° long-way, TOF ≈ 4.71. The GUI reports $z \approx 5.55$, $a \approx 1.00$, $e \approx 0$, $p \approx 1.00$.

Cases 3–4: direction handling (CW/CCW). Changing `tm` flips the transfer direction through the plane-normal selection, producing the expected sign change in tangential velocity vectors while preserving the same scalar solution branches (same z and same circular (a, p) up to rounding). This matches the GUI behavior shown in Figs. 3–4.

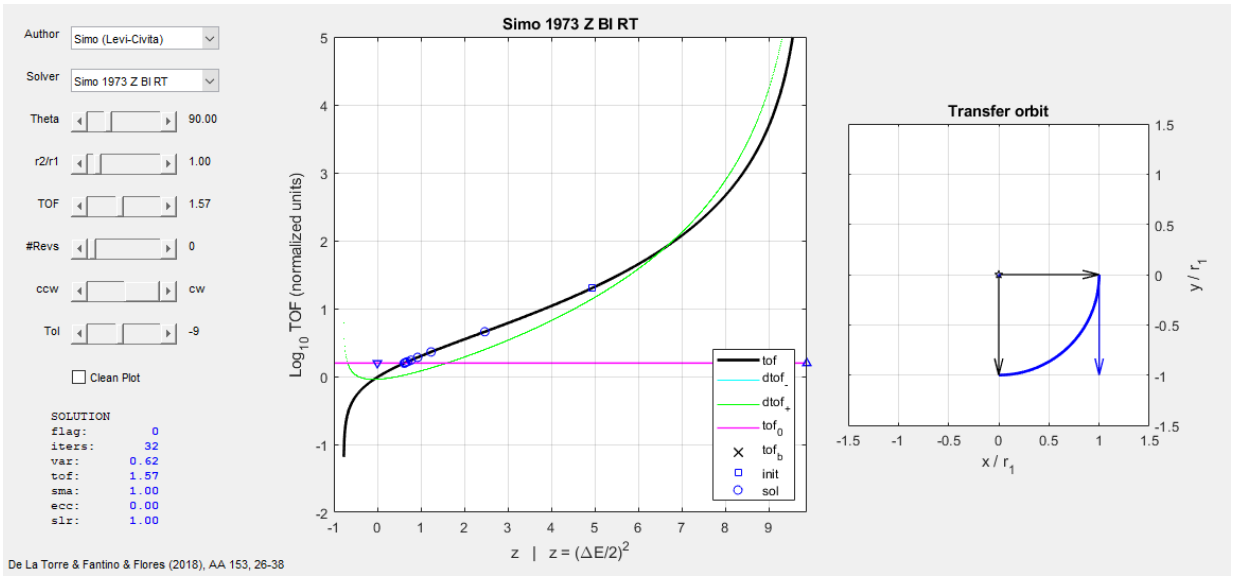


Figure 3. GUI reference – Case 3: direction-flipped circular solution (same scalar branch, opposite transverse direction).

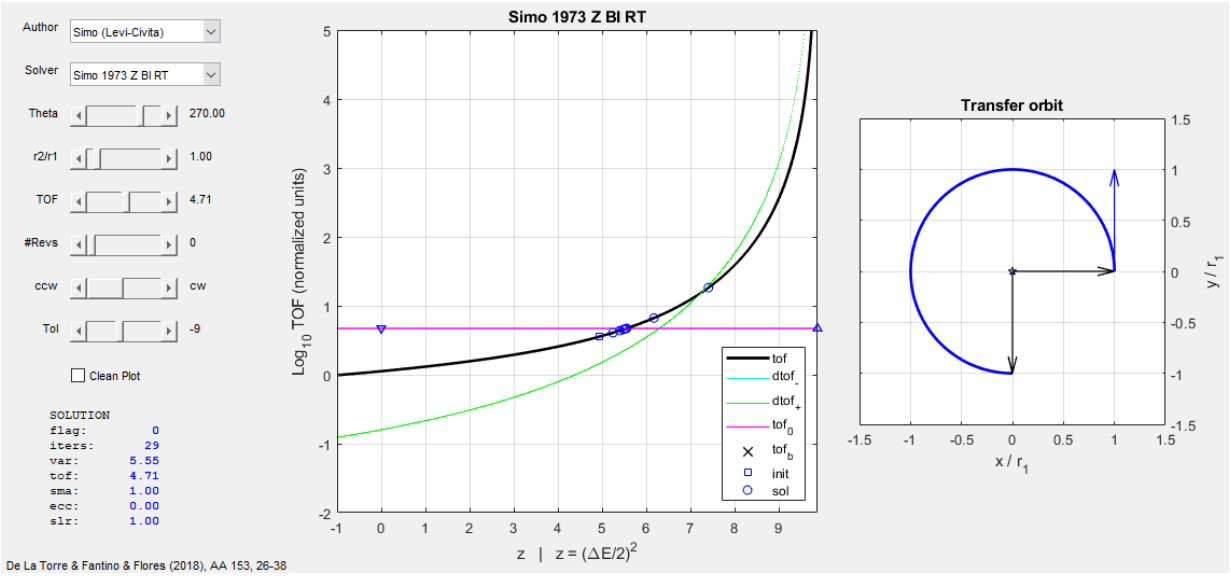


Figure 4. GUI reference – Case 4: direction-flipped long-way circular solution (same scalar branch, opposite transverse direction).

Case 5: 90° **rectilinear-like transfer (very small TOF).** For $\Delta t = 0.01$ (canonical time units), the solver transitions to the hyperbolic branch:

$$z = -0.7766947630, \quad a \approx -5.0 \times 10^{-5}, \quad p \approx 1.0001 \times 10^{4}, \quad e \approx 1.4142 \times 10^{4}. \tag{42}$$

The resulting velocities are large in magnitude (v1 = [-99.99, -100.00]), consistent with the GUI readout (Fig. 5). The radial consistency and time residual remain close to machine precision (`r_check_err`$\sim 10^{-16}$, `dt_err`$\sim 10^{-12}$), confirming robust convergence in an extreme setting. Although $\Delta t = 0.01$ is the minimum able to select in the GUI, the solver is able to compute the values for a $\Delta t = 0$.

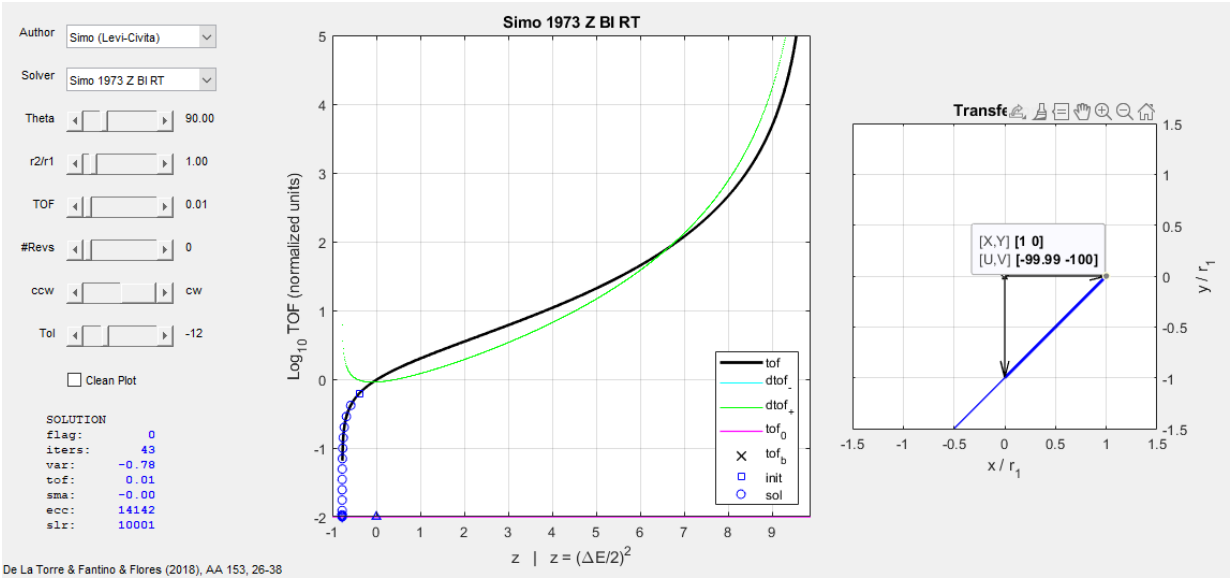


Figure 5. GUI reference – Case 5: rectilinear-like transfer (very small TOF). The GUI reports $z \approx -0.78$, $a \approx -0.00$, $e \approx 1.4142 \times 10^{4}$, $p \approx 1.0001 \times 10^{4}$.

Hohmann transfer (collinear endpoints). For $\mathbf{r}_1 = [1,0,0]$, $\mathbf{r}_2 = [-2,0,0]$ and the half-period time of flight $\Delta t = \pi\sqrt{a^3/\mu}$ with $a = (1+2)/2 = 1.5$, the solver returns

$$\Delta\theta = 180^\circ, \quad z = 2.4674011003, \quad a = 1.5, \quad p = \frac{4}{3}, \quad e = \frac{1}{3}, \tag{43}$$

with `dt_err`$\approx 6.53\times10^{-13}$ and `r_check_err`$\approx 1.11\times10^{-15}$. Velocity magnitudes match the analytical Hohmann values: $\|\mathbf{v}_1\| = \sqrt{4/3} \approx 1.1547$ and $\|\mathbf{v}_2\| = \sqrt{1/3} \approx 0.57735$, consistent with the GUI display (Fig. 6).

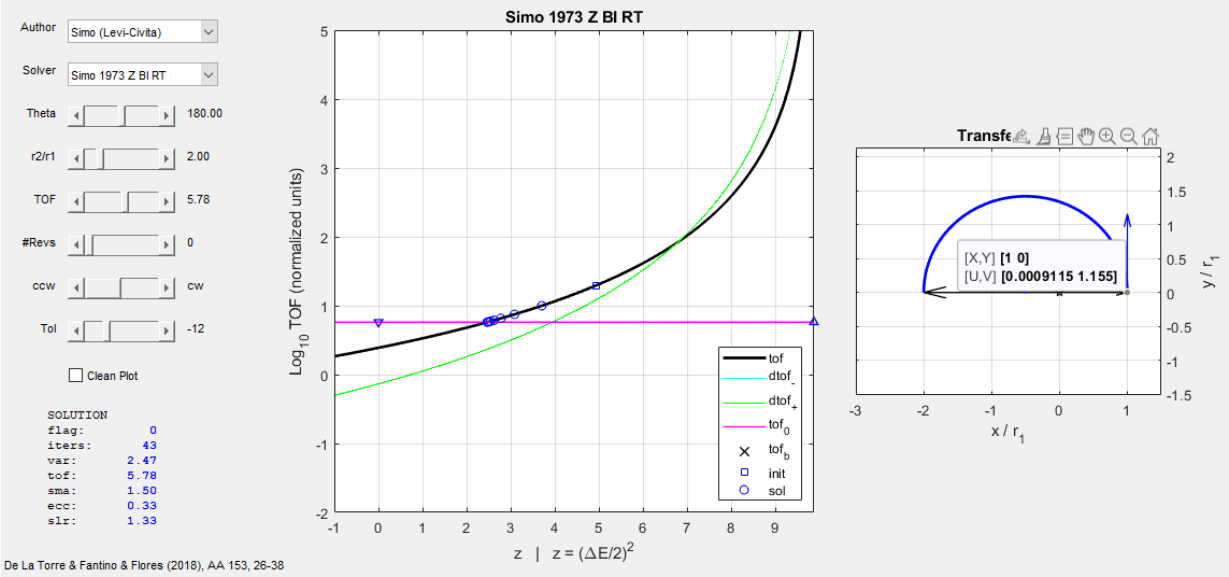


Figure 6. GUI reference – Hohmann transfer (180°, $r_2/r_1 = 2$), TOF ≈ 5.78. The GUI reports $z \approx 2.47$, $a \approx 1.50$, $e \approx 0.33$, $p \approx 1.33$.

Table 2. Canonical validation for the Simó Lambert solver ($\mu = 1$). GUI values are read from the reference screenshots; Python values are from the printed output.

Case	$\Delta\theta$ [deg]	TOF [-]	GUI (approx.) z	GUI (approx.) a	GUI (approx.) e	GUI (approx.) p	Python output z	Python output a	Python output e	Python output p	`dt_err`	`r_check_err`
1: 90 short-way	90	1.57	0.62	1.00	0.00	1.00	0.6168502751	1.0000	≈ 0	1.0000	2.94×10^{-13}	0
2: 270 long-way	270	4.71	5.55	1.00	0.00	1.00	5.5516524756	1.0000	≈ 0	1.0000	6.53×10^{-13}	0
3: dir. flip	90	1.57	0.62	1.00	0.00	1.00	0.6168502751	1.0000	≈ 0	1.0000	2.94×10^{-13}	0
4: dir. flip	270	4.71	5.55	1.00	0.00	1.00	5.5516524756	1.0000	≈ 0	1.0000	6.53×10^{-13}	0
5: rectilinear	90	0.01	-0.78	-0.00	14142	10001	-0.7766947630	-5.0e-5	14142	10000.7535	3.97×10^{-12}	4.44×10^{-16}
Hohmann (180)	180	5.78	2.47	1.50	0.33	1.33	2.4674011003	1.5000	0.3333	1.3333333333	6.53×10^{-13}	1.11×10^{-15}

Interpretation and consistency of the canonical results. Across Cases 1–4, the solver recovers a *unit circular orbit* as expected. This is evidenced by the conic parameters

$$a \approx 1, \qquad p \approx 1, \qquad e \approx 0, \tag{44}$$

which uniquely describe a circle of radius $r = 1$ in canonical units. For a circular orbit, the velocity magnitude is constant and purely tangential,

$$\|\mathbf{v}\| = \sqrt{\frac{\mu}{r}} = \sqrt{\frac{1}{1}} = 1, \qquad v_r = 0, \tag{45}$$

which matches the printed vectors: $\mathbf{v}_1 = [0, \pm 1, 0]$ and $\mathbf{v}_2 = [\mp 1, 0, 0]$. Geometrically, this corresponds to traversing the same circumference (same underlying circle) between the two radius vectors, only differing by the selected arc length.

While (a, p, e) remain those of the unit circle, the auxiliary Simó parameter z *does change* between the short-way and long-way solutions ($z \approx 0.617$ vs $z \approx 5.552$). This is consistent with the Simó formulation: z is not itself a direct conic element, but a universal-like parameter that selects the appropriate branch of the time-of-flight equation for the given $(r_1, r_2, \Delta\theta, \Delta t)$. Therefore, different arcs on the same circle can correspond to different z values while yielding identical physical conic parameters.

Cases 3–4 validate the direction handling. Changing the direction flag `tm` reverses the chosen transfer-plane normal and thus flips the transverse basis vectors $\hat{\mathbf{t}}_i$. As a result, the tangential velocity components change sign (prograde vs retrograde motion) while the scalar solution branch (and hence z, a, p) remains invariant up to numerical tolerance. This is the expected symmetry of the Lambert boundary-value problem under time-reversal/direction reversal in the two-body model.

Case 5 ("rectilinear") probes an extreme small time of flight ($\Delta t = 0.01$), for which the transfer must be a very high-energy solution. The solver correctly selects the hyperbolic branch ($z < 0$) and returns a negative semi-major axis ($a < 0$), which is a defining property of hyperbolic conics. The very large velocity magnitudes (order 10^2 in canonical units) are physically consistent: to satisfy the same geometric endpoints in a drastically shorter Δt, the trajectory requires a much larger specific orbital energy. The GUI readout reports the same qualitative regime (large e and large p), and the Python implementation matches these values within the displayed precision.

Finally, the Hohmann case validates the solver in the collinear geometry ($\Delta\theta = 180^\circ$), where the transfer plane is not uniquely defined by $\mathbf{r}_1 \times \mathbf{r}_2$. By supplying `plane_normal`, the solver selects a consistent plane and recovers the analytical Hohmann conic:

$$a = \frac{r_1 + r_2}{2} = 1.5, \qquad e = \frac{r_2 - r_1}{r_1 + r_2} = \frac{1}{3}, \qquad p = a(1 - e^2) = \frac{4}{3}. \tag{46}$$

The computed endpoint speed magnitudes, $\|\mathbf{v}_1\| \approx \sqrt{4/3}$ and $\|\mathbf{v}_2\| \approx \sqrt{1/3}$, match the known Hohmann values. Together with time-of-flight residuals of order 10^{-12}–10^{-13} and radial consistency errors near machine precision, these checks confirm that the implementation reproduces the GUI reference and behaves consistently across elliptic and hyperbolic regimes.

2.3.2 Multi-Revolution Extension and Validation

The Lambert solver has been updated to support multi-revolution transfers ($N \geq 1$). While the single-revolution algorithm assumes the transfer occurs within the shortest time for a given geometry, multi-revolution solutions allow the spacecraft to complete n_{rev} full orbital periods before arriving at $\mathbf{r}_2$.

Updated Time-of-Flight Equation. To accommodate these cases, the time-of-flight equation $\Delta t(z)$ was modified. The total duration is the sum of the base Simó transfer time (Eq. 29) and the time required for n_{rev} complete revolutions.

The specific term added to the formulation is derived from the period equation $T = 2\pi\sqrt{a^3/\mu}$, expressed in terms of the Simó variables z, P, Q. The implemented equation is:

$$\Delta t(z) = \Delta t_0(z) + \frac{\pi \cdot n_{rev}}{c_1(z)^3} \sqrt{\frac{1}{2\mu}} \left[\frac{P - Q \cdot c_0(z)}{z} \right]^3, \tag{47}$$

where $\Delta t_0(z)$ represents the single-revolution time computed in Eq. (29). This formulation allows the root-finding algorithm to seamlessly solve for z across both single and multi-revolution domains.

Validation against GUI Reference ($N = 1$). Unlike single-revolution transfers, multi-revolution cases for a fixed time of flight often yield two distinct solutions (dual branches), corresponding to different semi-major axes. To validate this behavior, a test case was executed with $\Delta t = 10$ (canonical units) and $N = 1$, using the standard endpoints but requesting a 270° transfer (CW).

The Python solver correctly identified both the "Left" and "Right" solution branches. As shown in Table 3, the solver recovered two distinct sets of orbital elements. The *Left Branch* yields a higher energy orbit ($a \approx 1.22$), while the *Right Branch* yields a lower energy orbit ($a \approx 0.95$). Both solutions satisfy the timing constraint with residuals $\mathcal{O}(10^{-12})$. Figure 7 confirms the qualitative behavior of the multi-revolution geometry.

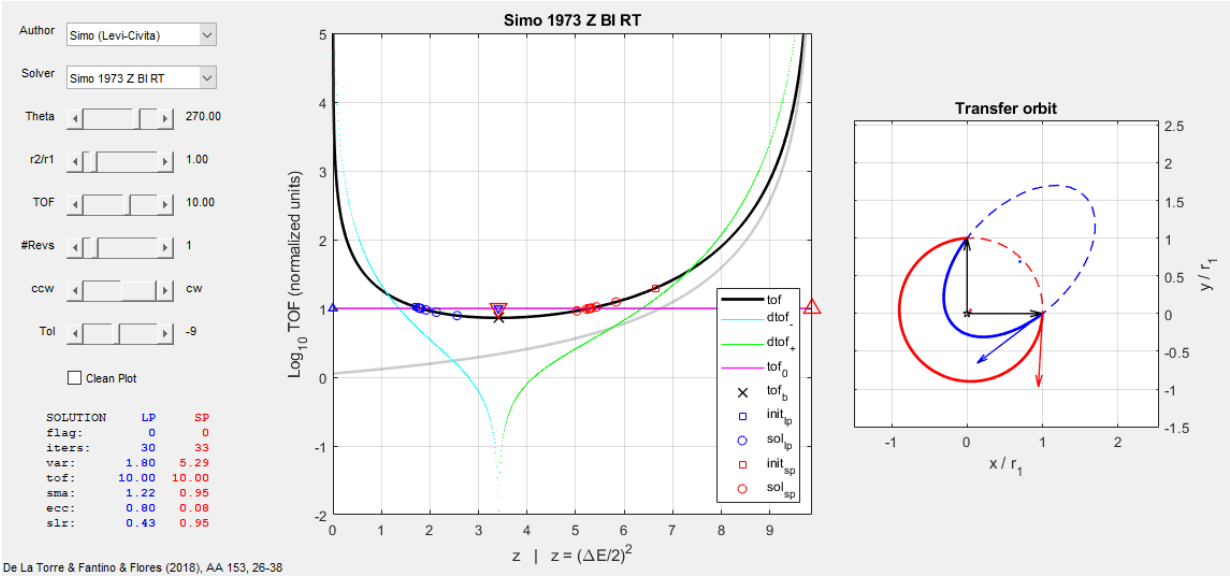


Figure 7. Validation of Multi-revolution Case ($N = 1$): $\Delta t = 10$, $\Delta\theta = 270°$. The solver correctly identifies two distinct valid trajectories (Left and Right branches) for the same flight time.

Table 3. Numerical results for Multi-Revolution Test ($N = 1, \Delta t = 10$). The solver brackets and converges on both possible branches.

Branch	$\Delta\theta$ [deg]	TOF [-]	z [-]	a [-]	p [-]	dt_err [-]	r_chk_err [-]
Left (CW)	270.0	10.0	1.80340	1.2221	0.4311	5.76×10^{-13}	0.0
Right (CW)	270.0	10.0	5.29365	0.9517	0.9462	1.15×10^{-12}	4.44×10^{-15}

Table 4. GUI results for Multi-Revolution Test ($N = 1, \Delta t = 10$).

Branch	$\Delta\theta$ [deg]	TOF [-]	z [-]	a [-]	p [-]
Left (CW)	270.0	10.0	1.80	1.22	0.43
Right (CW)	270.0	10.0	5.29	0.95	0.95

When comparing the results given by the developed script and the GUI, the only difference lies in the precision of the decimals that are displayed in the user interface, our results being as precise as possible.

3 PORK-CHOP PLOT TOOL FOR LAUNCH WINDOW SEARCH

3.1 Introduction

The overall objective of the course project is to build an interplanetary mission analysis tool able to trade off different departure and arrival dates and to compute the corresponding ΔV budget. In heliocentric mission design, this trade space is commonly explored with a *pork-chop plot* (PCP), which visualizes the transfer cost over a grid of departure epochs and times of flight [11]. The PCP is particularly useful for identifying launch windows, quantifying the sensitivity of ΔV to timing, and comparing alternative transfer directions (e.g., CCW vs. CW motion) using the same Lambert boundary-value solver [11].

Given a departure planet and an arrival planet, the two-body heliocentric Lambert problem provides the spacecraft endpoint velocities that connect two heliocentric position vectors $\mathbf{r}_1$ and $\mathbf{r}_2$ in a prescribed time of flight Δt (Section 2) [9], [12]. The pork-chop plot extends this by scanning many Lambert problems, where each grid point is defined by

$$t_{\text{dep}} \in [t_0, t_1], \qquad \Delta t \in [\Delta t_{\min}, \Delta t_{\max}], \qquad t_{\text{arr}} = t_{\text{dep}} + \Delta t. \tag{48}$$

At each $(t_{\text{dep}}, \Delta t)$, the departure and arrival planetary states are evaluated from the ephemeris model (e.g., Horizons conventions) [13],

$$\left(\mathbf{r}_{\text{dep}}, \mathbf{v}_{\text{dep}}\right) = \left(\mathbf{r}_{\text{pl}}(t_{\text{dep}}), \mathbf{v}_{\text{pl}}(t_{\text{dep}})\right), \qquad \left(\mathbf{r}_{\text{arr}}, \mathbf{v}_{\text{arr}}\right) = \left(\mathbf{r}_{\text{pl}}(t_{\text{arr}}), \mathbf{v}_{\text{pl}}(t_{\text{arr}})\right), \tag{49}$$

and a Lambert solver returns the corresponding heliocentric spacecraft velocities $\mathbf{v}_{\text{sc},1}$ (at departure) and $\mathbf{v}_{\text{sc},2}$ (at arrival) [9], [12]. The project requirement to compute $\Delta V_T = |\Delta V_1| + |\Delta V_2|$ is implemented by evaluating the heliocentric velocity mismatches

$$\Delta V_T(t_{\text{dep}}, \Delta t) = \left\|\mathbf{v}_{\text{sc},1} - \mathbf{v}_{\text{dep}}\right\| + \left\|\mathbf{v}_{\text{sc},2} - \mathbf{v}_{\text{arr}}\right\|. \tag{50}$$

This quantity is an effective *screening metric* for launch opportunity search in preliminary, patched two-body survey tools [11].

In addition to the 2D PCP heatmap, the developed tool automatically extracts the minimum ΔV_T solution found in the scan and visualizes the associated transfer arc together with the planetary trajectories in a 3D heliocentric plot. This satisfies the project requirement to "generate the PCP and plot the transfer orbit" and provides a qualitative check of the computed optimum.

3.2 Methodology

3.2.1 Software Entry Point and Function Interface

The pork-chop tool is implemented as the function `porkchop_plot(departure_planet, arrival_planet, ..`
Its key inputs map directly to the mission-analysis degrees of freedom:

- **Planet identifiers:** `departure_planet` and `arrival_planet` select the ephemerides queried through `planet_state_jd(...)`.
- **Departure-date window:** either a manual interval `dep_range` or an automatically sized window specified in `synodic_periods` starting at `launch_window_start_date`.
- **Time-of-flight range:** `min_tof` and `max_tof` define the TOF scan in days, consistent with the Julian-date convention used internally [3].
- **Grid resolution:** `res` sets the number of samples in each dimension, producing a `res` $\times$ `res` Lambert scan.
- **Transfer direction:** `tm` is forwarded to the Lambert solver and selects the directed transfer angle convention (CCW vs. CW) as described in Section 2.

- **Visualization controls:** `show_heatmap`, `show_contours`, and `dv_max` control the appearance and the masking of high-cost solutions.

3.2.2 Departure Window Determination (Manual vs. Synodic-Based)

The code supports two strategies to define the departure-date interval $[t_0, t_1]$:

(i) Synodic-period based window. If `synodic_periods` is provided, the code estimates the synodic period S from approximate orbital periods P_1 and P_2 (in days) stored in `PLANET_PERIODS`. The standard synodic relation is implemented:

$$\frac{1}{S} = \left| \frac{1}{P_1} - \frac{1}{P_2} \right|, \qquad S = \left(\left| \frac{1}{P_1} - \frac{1}{P_2} \right| \right)^{-1}, \tag{51}$$

with a trivial fallback $S = P_1$ when $P_1 = P_2$. The departure window length is then

$$\Delta t_{\text{win}} = S \cdot N_{\text{syn}}, \tag{52}$$

where $N_{\text{syn}} = \texttt{synodic_periods}$. The start date is converted from UTC calendar format to Julian Date using `JD_from_UTC` [3]:

$$t_0 = \text{JD}(\texttt{launch_window_start_date}), \qquad t_1 = t_0 + \Delta t_{\text{win}}. \tag{53}$$

This option is convenient for scanning multiple repeating launch opportunities without manually specifying end dates.

(ii) Manual date range. If `synodic_periods` is not provided, the user supplies `dep_range` as two UTC calendar tuples. The code converts them to Julian Dates and uses

$$t_0 = \text{JD}(\texttt{dep_range}[0]), \qquad t_1 = \text{JD}(\texttt{dep_range}[1]). \tag{54}$$

This option is suited for course-mandated fixed windows (e.g., 2020–2025).

3.2.3 Grid Generation and Data Structures

After defining the window, the tool generates the two scan axes:

Departure dates (PCP vertical axis).

$$t_{\text{dep},i} = \text{linspace}(t_0, t_1, \texttt{res}), \qquad i = 1, \ldots, \texttt{res}. \tag{55}$$

Times of flight (PCP horizontal axis).

$$\Delta t_j = \text{linspace}(\Delta t_{\min}, \Delta t_{\max}, \texttt{res}), \qquad j = 1, \ldots, \texttt{res}. \tag{56}$$

The computed cost values are stored in a 2D array `total_dv` of shape $(\texttt{res}, \texttt{res})$ such that `total_dv[i,j]` corresponds to the pair $(t_{\text{dep},i}, \Delta t_j)$. The array is initialized with `NaN` to mark uncomputed or invalid points; this choice is deliberate because Matplotlib naturally omits `NaN` values in `pcolormesh` and contouring, creating "white" regions where no feasible solution is available or where solutions are intentionally masked.

In addition, two scalar trackers are maintained:

- `min_dv`: the minimum ΔV_T observed in the scan,
- `best_transfer`: a dictionary storing the departure/arrival epochs and states for the current best grid point (used later for the 3D trajectory plot).

3.2.4 Trajectory Scan Loop and Lambert Calls

The computational core is a nested loop over the grid:

1. For each departure epoch $t_{\mathrm{dep},i}$, query the departure planet state in heliocentric coordinates:

$$(\mathbf{r}_{\mathrm{dep}}, \mathbf{v}_{\mathrm{dep}}) = \texttt{planet_state_jd}(\texttt{departure_planet}, t_{\mathrm{dep},i}). \tag{57}$$

   In the present implementation, `in_km=False` indicates the canonical internal units AU and AU/day (consistent with $\mu_\odot$ in $\mathrm{AU}^3/\mathrm{day}^2$).

2. For each time of flight Δt_j, compute the arrival epoch $t_{\mathrm{arr}} = t_{\mathrm{dep},i} + \Delta t_j$ and query the arrival planet state:

$$(\mathbf{r}_{\mathrm{arr}}, \mathbf{v}_{\mathrm{arr}}) = \texttt{planet_state_jd}(\texttt{arrival_planet}, t_{\mathrm{arr}}). \tag{58}$$

3. Solve the Lambert boundary-value problem under the Sun's gravitational parameter:

$$(\mathbf{v}_{\mathrm{sc},1}, \mathbf{v}_{\mathrm{sc},2}) = \texttt{lambert_simo}(\mu_\odot, \mathbf{r}_{\mathrm{dep}}, \mathbf{r}_{\mathrm{arr}}, \Delta t_j; \texttt{tm}). \tag{59}$$

   The direction flag `tm` ensures that the directed transfer angle is consistent with the requested CCW/CW motion, as validated in the Lambert section. The solver returns `None` or raises an exception for invalid geometries or out-of-domain evaluations; such cases are skipped and remain as `NaN` in the grid.

This brute-force scan is intentionally transparent and directly aligned with the course requirement to explain the main software functions. Its computational cost scales as $\mathcal{O}(\texttt{res}^2)$ Lambert solves. For example, `res=500` corresponds to 2.5×10^5 grid points, which is why the code prints progress information and uses exception handling to prevent single failures from stopping the scan.

3.2.5 Delta-V Metric and Optional Masking

Once a Lambert solution is available, the code computes the two impulse mismatches:

$$\Delta\mathbf{v}_1 = \mathbf{v}_{\mathrm{sc},1} - \mathbf{v}_{\mathrm{dep}}, \tag{60}$$

$$\Delta\mathbf{v}_2 = \mathbf{v}_{\mathrm{sc},2} - \mathbf{v}_{\mathrm{arr}}, \tag{61}$$

and evaluates the total cost ΔV_T as in Eq. (50). Because the internal states are expressed in AU/day, the magnitudes are converted to km/s using `au_per_day_to_km_per_s` before being stored and displayed.

To improve plot readability and to focus on physically meaningful windows, the parameter `dv_max` implements a hard mask:

$$\text{if } \Delta V_T > \Delta V_{\max} \text{ then store NaN, else store } \Delta V_T. \tag{62}$$

This removes very-high-energy solutions that would otherwise dominate the color scale and wash out the low-cost region of interest.

3.2.6 Tracking the Minimum Solution

During the scan, the code maintains the best (minimum) cost found so far:

$$\Delta V_T < \Delta V_{T,\min} \;\Rightarrow\; \text{update } \Delta V_{T,\min} \text{ and store the associated states.} \tag{63}$$

The stored dictionary includes: `t_dep`, `t_arr`, `dt`, `dv_tot`, the endpoint planet states `r_dep`, `v_dep`, `r_arr`, `v_arr`, and the Lambert departure velocity `v_transfer_start`. These quantities are later reused to generate the 3D plot without recomputing the Lambert solution.

3.2.7 Visualization: 2D PCP Heatmap and Contours

The left subplot is constructed using a meshgrid in which:

- the x-axis is Δt in days (`tof_days`),
- the y-axis is the departure date expressed in *years from the window start* for readability:

$$y_i = \frac{t_{\text{dep},i} - t_0}{365.25}. \tag{64}$$

The heatmap is rendered via `pcolormesh`, with `NaN` values automatically omitted. Optionally, isolines of constant ΔV_T are overlaid using `contour` and labeled. Finally, the minimum-cost grid point is marked with a distinct symbol and a legend entry ("Min dV").

3.2.8 Visualization: 3D Geometry and Transfer Orbit Reconstruction

To satisfy the requirement of plotting the transfer orbit, the right subplot shows a 3D heliocentric view of:

- the Sun (at the origin),
- a short segment of the departure planet trajectory around the transfer window,
- a short segment of the arrival planet trajectory,
- the transfer trajectory corresponding to the minimum ΔV_T solution.

The planetary paths are generated by sampling the ephemeris model over a time span around t_{dep} and t_{arr}. The spacecraft path is reconstructed by propagating the two-body motion from $(\mathbf{r}_{\text{dep}}, \mathbf{v}_{\text{sc},1})$ using `propagate_lagrange` for times $\tau \in [0, \Delta t]$ [3]. This produces a discrete transfer arc consistent with the Lambert endpoint conditions and provides an intuitive geometric interpretation of the optimum.

To avoid misleading visual distortion, the axes limits are set using a symmetric range about the midpoint of all plotted planetary points. This approximates an equal-aspect 3D view, ensuring the orbital shapes are visually comparable across axes.

3.3 Results

The developed Pork Chop Plot (PCP) tool evaluates the total heliocentric ΔV required to transfer from Earth to a target planet by scanning a 2D grid of *departure date* t_{dep} and *time of flight* Δt. This type of launch-window survey and visualization is standard in preliminary interplanetary mission design and is commonly used to reveal low-cost regions in $(t_{\text{dep}}, \Delta t)$ space [11]. For each grid point, the planetary heliocentric states $(\mathbf{r}_{\text{dep}}, \mathbf{v}_{\text{dep}})$ and $(\mathbf{r}_{\text{arr}}, \mathbf{v}_{\text{arr}})$ are computed at t_{dep} and $t_{\text{arr}} = t_{\text{dep}} + \Delta t$ using the ephemeris model implemented in `planet_state_jd`, which follows the standard concept of ephemeris-based state retrieval (e.g., as provided by JPL Horizons) [13]. The Lambert boundary-value problem is then solved for the same endpoints and Δt, returning transfer velocities $\mathbf{v}_{1,\text{sc}}$ and $\mathbf{v}_{2,\text{sc}}$ at departure and arrival [9], [12].

The displayed cost is the sum of the magnitudes of the heliocentric velocity mismatches at the endpoints (equivalent to Eq. (50)):

$$\Delta V_T = \|\Delta \mathbf{v}_1\| + \|\Delta \mathbf{v}_2\| = \|\mathbf{v}_{1,\text{sc}} - \mathbf{v}_{\text{dep}}\| + \|\mathbf{v}_{2,\text{sc}} - \mathbf{v}_{\text{arr}}\|, \tag{65}$$

converted from AU/day to km/s for reporting. Using a Lambert-based heliocentric mismatch as a screening metric is consistent with the patched two-body / preliminary survey assumptions typically employed in pork-chop tools [11]. In each PCP figure, the left panel shows the $\Delta V_T(t_{\text{dep}}, \Delta t)$ field as a heatmap with overlaid isolines. Such isoline/contour overlays are a standard way to interpret pork-chop plots and to locate low-cost launch opportunities [11]. A marker indicates the minimum ΔV_T found within the scanned domain.

The right panel visualizes the corresponding *optimal trajectory* in 3D: the dashed arcs represent short segments of the departure and arrival planet heliocentric orbits around the optimum dates, while the solid curve is the Lambert transfer propagated from the optimal departure state. Although the PCP is a 2D map in $(t_{\text{dep}}, \Delta t)$, the transfer and planetary motion are fully three-dimensional because ephemeris states include out-of-plane components (inclination effects) [13]. The transfer arc is consistent with the Lambert endpoint conditions by construction [9], [12].

To facilitate comparison across targets, Table 5 summarizes the minima read directly from the figure headers (departure date and TOF of the highlighted optimum).

Table 5. Summary of the minimum ΔV_T solutions identified by the PCP scans (values from the plot titles). For long-flight cases, the TOF is additionally expressed in years.

Target	Direction	$\Delta V_{T,\text{min}}$ [km/s]	Dep. date	TOF [days]	TOF [years]
Mars	CCW	6.32	2020-07-24	205.2	0.56
Mars	CW	62.12	2023-03-11	113.0	0.31
Mercury	CCW	18.62	2020-04-26	87.7	0.24
Venus	CCW	7.56	2021-10-29	158.7	0.43
Jupiter	CCW	15.38	2021-05-12	865.7	2.37
Saturn	CCW	17.42	2020-04-03	1931.1	5.29
Uranus	CCW	16.42	2020-07-07	5339.7	14.62
Neptune	CCW	16.99	2021-06-27	19200.9	52.57
Pluto	CCW	16.53	2021-06-30	57346.7	157.00

3.3.1 Mars: comparison between CCW and CW transfers

Figures 8 and 9 show the Earth–Mars PCPs for two opposite transfer directions. In the CCW case (Fig. 8), a pronounced low-ΔV valley is visible in the scan domain. Such "valley" structures are the typical signature of favorable launch windows in pork-chop plots [11]. The repeated band structure in the vertical (departure-date) direction indicates that comparable transfer geometries reoccur as the relative phase between planets evolves, which is a standard qualitative feature of launch-window maps [11].

The 3D panel confirms that the optimum corresponds to a smooth heliocentric transfer arc connecting the planetary positions at departure and arrival, consistent with the Lambert two-point boundary-value formulation [9], [12]. Because the planetary states are taken from a full ephemeris model, the geometry includes the out-of-plane component implied by the ephemeris inclination [13].

In contrast, the CW case (Fig. 9) yields significantly higher ΔV throughout the scanned domain. The strong directional sensitivity is expected because changing the directed transfer angle changes the Lambert solution branch and therefore the endpoint velocities required to meet the same boundary conditions [9].

Effect of transfer direction (CCW vs. CW). For the Mars example in Table 5, changing the transfer direction from CCW to CW increases the minimum cost from 6.32 km/s to 62.12 km/s, i.e.,

$$\Delta(\Delta V) = 62.12 - 6.32 = 55.80 \text{ km/s}, \qquad \frac{\Delta V_{\text{CW}}}{\Delta V_{\text{CCW}}} \approx \frac{62.12}{6.32} \approx 9.83.$$

This illustrates that the direction-of-motion choice can dominate the ΔV budget in unfavorable geometries.

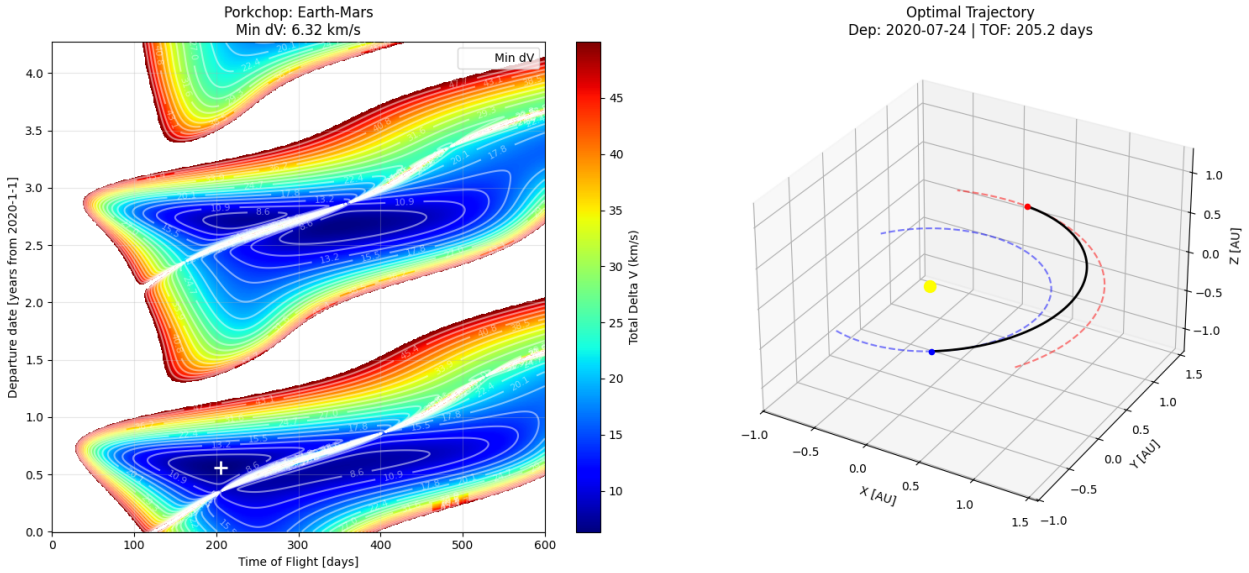


Figure 8. Earth–Mars PCP (CCW).

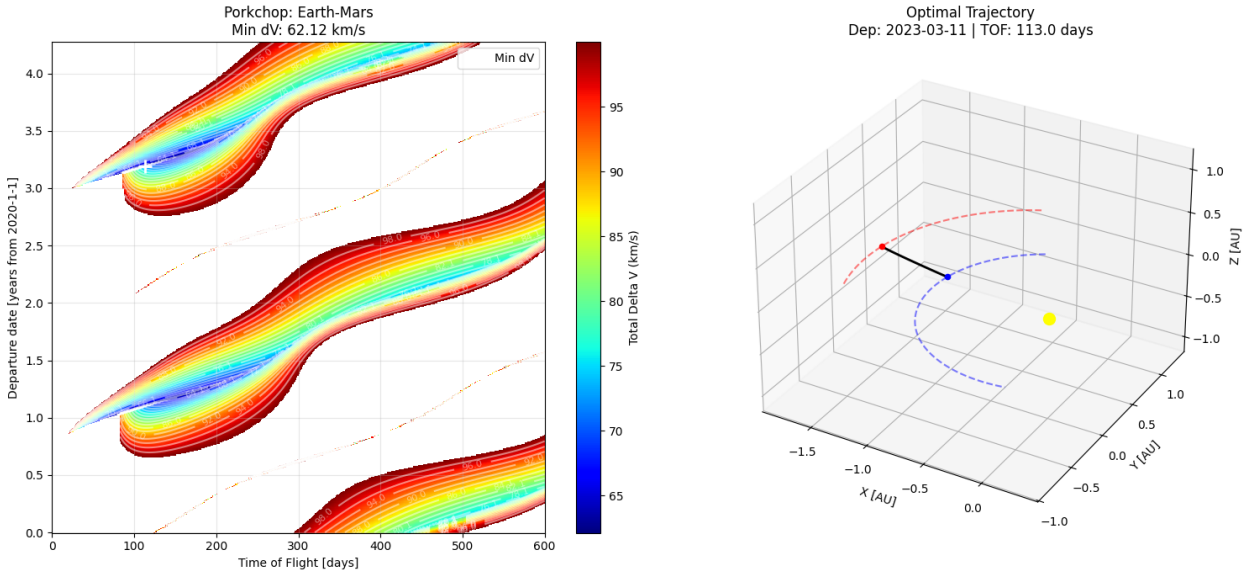


Figure 9. Earth–Mars PCP (CW).

3.3.2 Mercury: narrow launch opportunities and high-energy inner transfer

Figures 10 and 11 show the Earth–Mercury CCW PCP and a zoomed-in view around the optimum. The low-ΔV region is comparatively narrow, demonstrating strong sensitivity to small changes in departure date or time of flight. This "sharp valley" behavior is a common pork-chop characteristic when favorable solutions occupy a small subset of the $(t_{\text{dep}}, \Delta t)$ domain [11]. The 3D plot again reflects the full ephemeris geometry used to evaluate the boundary states [13], while the transfer arc satisfies the Lambert endpoint constraints [9], [12].

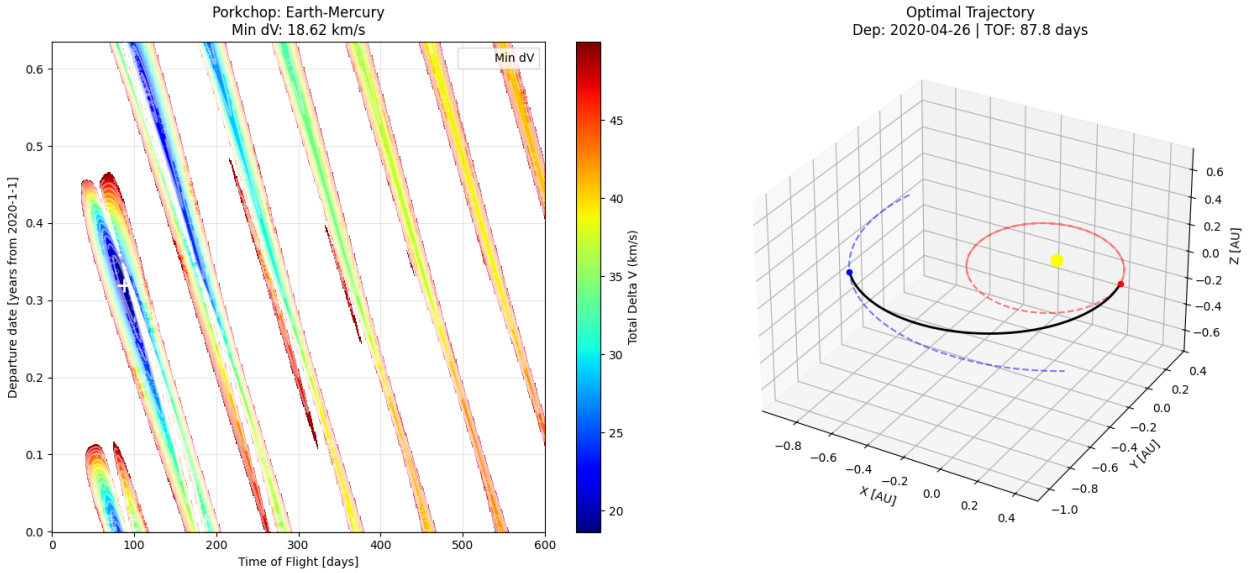


Figure 10. Earth–Mercury PCP (CCW).

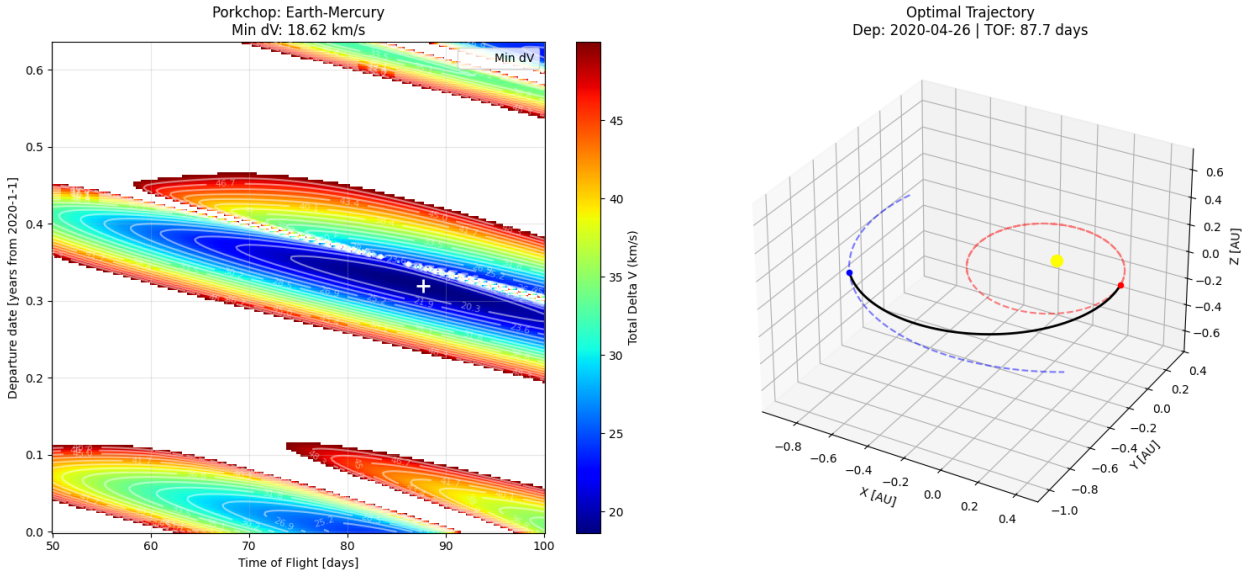


Figure 11. Earth–Mercury PCP (CCW), zoomed view.

3.3.3 Venus: low-energy inner transfer with broad feasible region

Figure 12 shows the Earth–Venus CCW PCP. The low-ΔV trough is broader than for Mercury, indicating a wider set of feasible departure/TOF combinations within the scanned domain. The interpretation of trough width as launch-window sensitivity is a standard qualitative reading of pork-chop maps [11]. The corresponding 3D geometry uses ephemeris states for the boundary conditions [13] and a Lambert transfer consistent with those endpoints [9], [12].

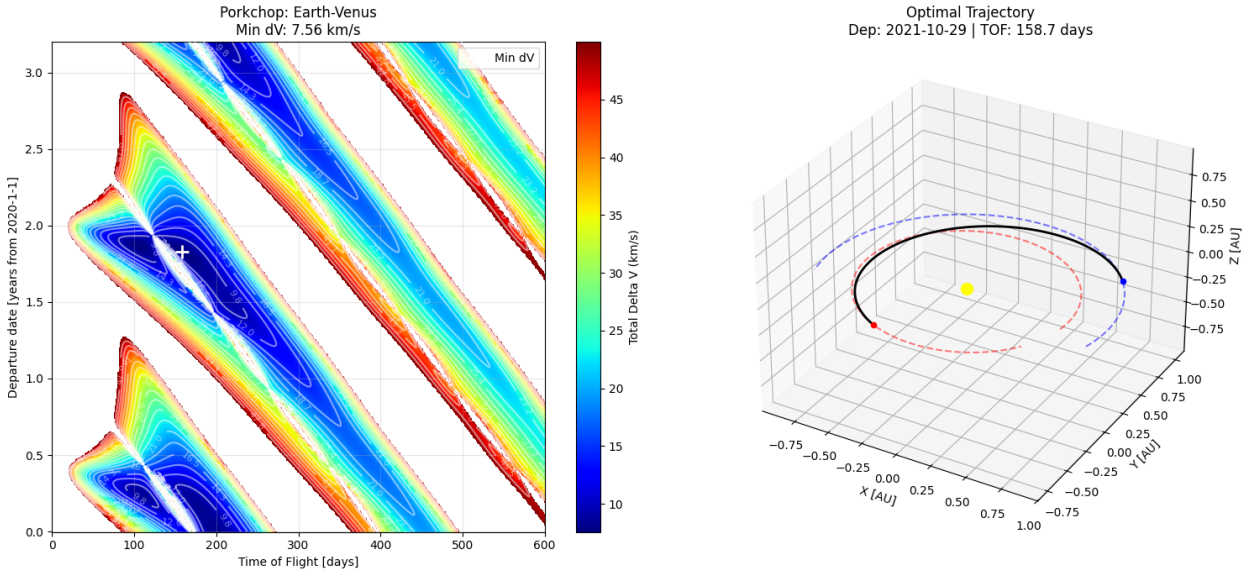


Figure 12. Earth–Venus PCP (CCW).

3.3.4 Jupiter: multi-year transfers and synodic repetition

Figure 13 presents the Earth–Jupiter CCW PCP. The low-cost region shifts to significantly larger times of flight, and repeating structures appear across the departure axis. Such repeated opportunity patterns are commonly observed in pork-chop surveys as the relative phase between planets evolves over time [11]. The 3D plot visualizes the corresponding Lambert transfer and ephemeris-based boundary states [9], [12], [13].

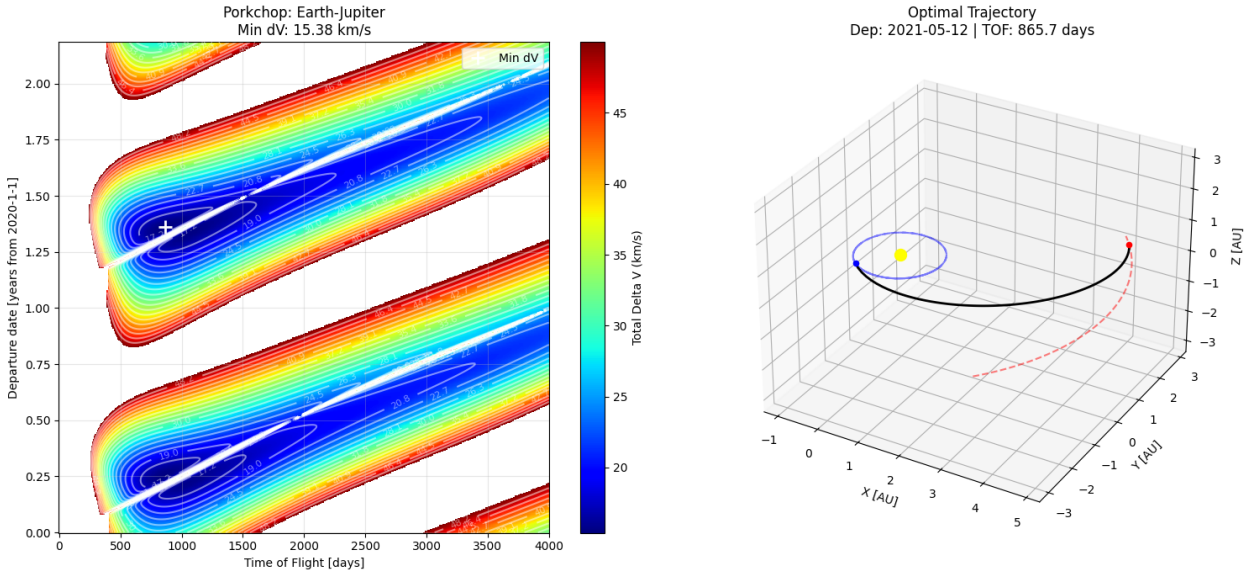


Figure 13. Earth–Jupiter PCP (CCW).

3.3.5 Saturn: longer TOF and increasing sensitivity to phasing

Figure 14 shows the Earth–Saturn CCW results. The minimum-cost region is located at longer TOF values, consistent with the overall scale increase visible in the 3D plot. The plotted transfer arc is obtained by enforcing the Lambert boundary conditions at the two ephemeris states [9], [12], [13].

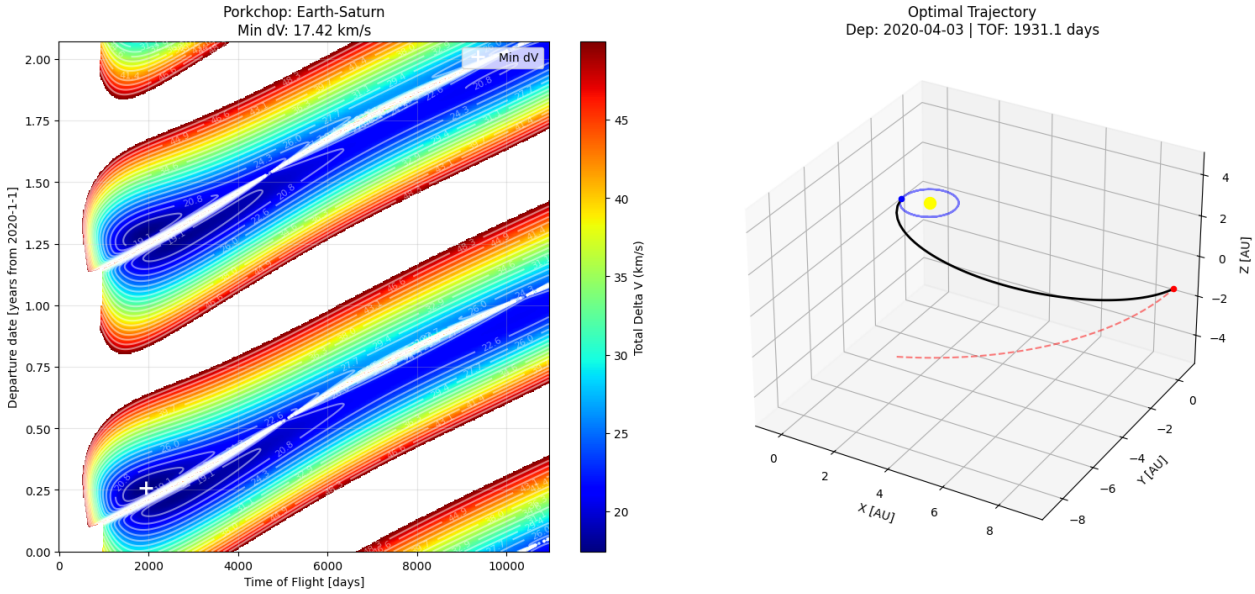


Figure 14. Earth–Saturn PCP (CCW).

3.3.6 Uranus, Neptune, and Pluto: decades-to-centuries direct transfers

Figures 15, 16, and 17 extend the PCP approach to the outermost targets. In all cases, the lowest-cost solutions within the explored domain occur at very large times of flight. This highlights a general limitation of purely ballistic, single-leg Lambert transfers for distant rendezvous when no intermediate assists are allowed, which is a standard outcome of preliminary pork-chop surveys [11]. The 3D visualizations show the large heliocentric scale of the corresponding Lambert arcs and the full three-dimensional ephemeris geometry [9], [12], [13].

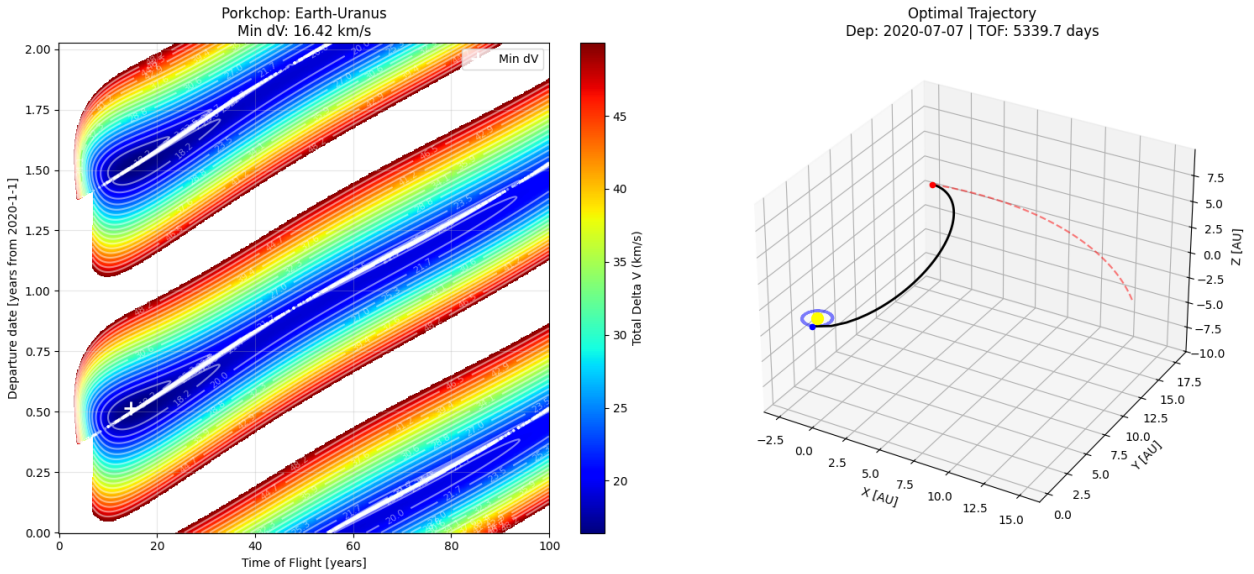


Figure 15. Earth–Uranus PCP (CCW).

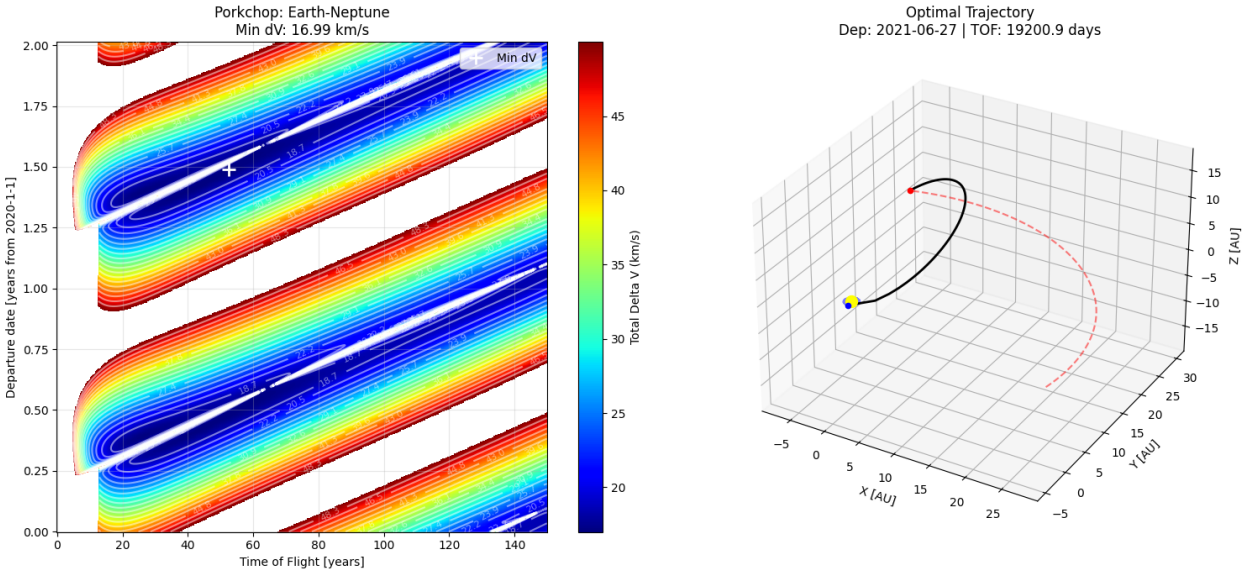


Figure 16. Earth–Neptune PCP (CCW).

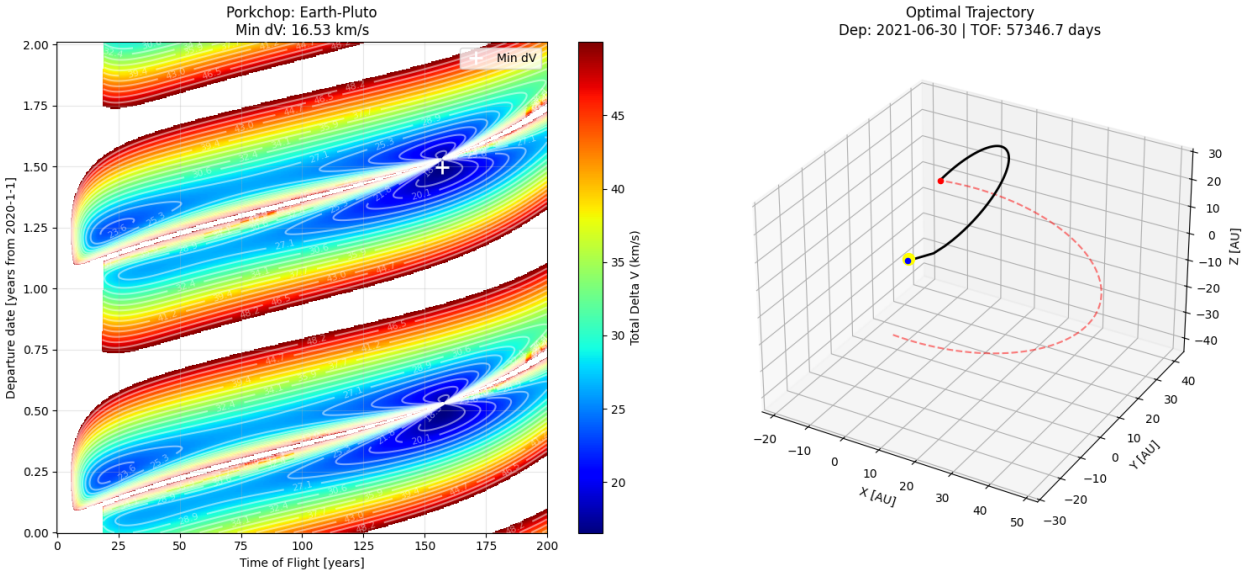


Figure 17. Earth–Pluto PCP (CCW).

4 MINIMUM ΔV AND TRANSFER-ORBIT ELEMENTS FOR A FIXED LAUNCH DATE

4.1 Introduction

Beyond visual porkchop plots (departure date vs. time of flight), mission design often requires a *compact numerical summary* for a *single launch date*: for each target planet, what is the minimum heliocentric impulsive cost ΔV_T and what are the corresponding transfer-orbit elements?

This section introduces a dedicated Python script that generates such a table. Given a fixed launch date t_{dep} (here chosen as the project deadline), the code scans candidate times of flight Δt to each planet and solves a Lambert boundary-value problem for every trial Δt. For each planet, the solution that minimizes the two-impulse patched-conic proxy

$$\Delta V_T = \|\Delta \mathbf{V}_1\| + \|\Delta \mathbf{V}_2\| = \|\mathbf{v}_{\text{tr},1} - \mathbf{v}_p(t_{\text{dep}})\| + \|\mathbf{v}_{\text{tr},2} - \mathbf{v}_p(t_{\text{arr}})\| \tag{66}$$

is retained, where $\mathbf{v}_{\text{tr},1}$ and $\mathbf{v}_{\text{tr},2}$ are the Lambert transfer velocities at departure and arrival, and $\mathbf{v}_p(\cdot)$ denotes the heliocentric velocity of the departure/arrival planets. This ΔV_T definition is the standard impulsive, Sun-centered two-body approximation used for preliminary interplanetary trade studies [2], [3].

In addition to ΔV_T, the script computes the Keplerian elements of the *transfer orbit at departure* from the heliocentric state $(\mathbf{r}_{\text{dep}}, \mathbf{v}_{\text{tr},1})$. These elements $(a, e, i, \Omega, \omega)$ provide an interpretable characterization of the selected conic (energy level via a, shape via e, and plane/orientation via i, Ω, ω) [3]. The Lambert solver itself is called via the previously developed `lambert_simo` routine; Lambert's problem and its multiple-solution structure are well documented in the astrodynamics literature [9], [10].

4.2 Methodology

4.2.1 Software Inputs, Units, and Conventions

The script imports ephemeris and Lambert utilities from the project module: `planet_state_jd`, `lambert_simo`, `JD_from_UTC`, `au_per_day_to_km_per_s`, and `mu_sun_AU`. Internally, the state vectors are handled in AU and AU/day, with the solar gravitational parameter μ expressed consistently in AU^3/day^2. Reported ΔV values are finally converted to km/s using the provided unit conversion utility.

A fixed launch date (y, m, d) is mapped to Julian Date t_{dep} via `JD_from_UTC`. For each trial time of flight Δt (in days), the arrival epoch is

$$t_{\text{arr}} = t_{\text{dep}} + \Delta t. \tag{67}$$

The heliocentric planetary states at t_{dep} and t_{arr} are obtained through `planet_state_jd`.

4.2.2 Lambert-Based ΔV_T Evaluation for a Fixed Launch Date

For a given target planet, the function `find_min_dv(target_planet, launch_date)` computes the Earth departure state and then scans candidate Δt values.

Step 1: Departure state (Earth). At t_{dep}, the Earth heliocentric state is $\mathbf{r}_E(t_{\text{dep}})$ and $\mathbf{v}_E(t_{\text{dep}})$, returned as `r_earth, v_earth`.

Step 2: Candidate TOF range from a Hohmann baseline. To avoid scanning an unnecessarily large TOF domain, the code uses approximate Earth-to-planet Hohmann times of flight (stored as a small lookup table) to define a *center* value Δt_H. The scan window is then chosen as

$$\Delta t \in [\max(20,\ 0.5\,\Delta t_H),\ 2.5\,\Delta t_H], \tag{68}$$

which is intended to cover both shorter-than-Hohmann and longer-than-Hohmann solutions in a coarse, robust way. The Hohmann transfer is the classical minimum-energy two-impulse transfer between coplanar circular orbits, and it provides a physically meaningful scale for expected interplanetary flight times [2], [3].

Step 3: Lambert solve and impulsive cost. For each candidate Δt in the grid, the code evaluates the target planet state $(\mathbf{r}_P(t_{\text{arr}}), \mathbf{v}_P(t_{\text{arr}}))$ and calls

$$(\mathbf{v}_{\text{tr},1}, \mathbf{v}_{\text{tr},2}) = \texttt{lambert_simo}(\mu_\odot,\ \mathbf{r}_E(t_{\text{dep}}),\ \mathbf{r}_P(t_{\text{arr}}),\ \Delta t,\ \texttt{tm} = +1). \tag{69}$$

The script currently fixes `tm=+1` (prograde / CCW branch selection in the transfer plane). If a Lambert call fails (e.g., numerical issues or invalid geometry), the candidate is skipped.

Whenever a solution is returned, the heliocentric two-impulse cost is computed as

$$\Delta\mathbf{V}_1 = \mathbf{v}_{\text{tr},1} - \mathbf{v}_E(t_{\text{dep}}), \tag{70}$$
$$\Delta\mathbf{V}_2 = \mathbf{v}_{\text{tr},2} - \mathbf{v}_P(t_{\text{arr}}), \tag{71}$$
$$\Delta V_T = \|\Delta\mathbf{V}_1\| + \|\Delta\mathbf{V}_2\|. \tag{72}$$

This is the standard preliminary ΔV proxy in a Sun-centered patched-conic setting, where maneuvers are treated as instantaneous heliocentric velocity changes at departure and arrival [2].

4.2.3 Two-Stage Search Strategy (Coarse + Local Refinement)

The minimization is performed with a simple yet effective two-stage grid search:

1. **Coarse scan:** 200 uniformly spaced TOF samples across the full window. The best candidate $\Delta t^\star$ is stored.
2. **Refinement scan:** a local re-scan around $\Delta t^\star$ using 100 samples in $\left[\Delta t^\star - \Delta t_{\text{span}},\ \Delta t^\star + \Delta t_{\text{span}}\right]$, with

$$\Delta t_{\text{span}} = \max(10,\ 0.1\,\Delta t^\star). \tag{73}$$

This approach trades optimality for speed and robustness: it is fully deterministic, requires no gradients, and is sufficient to populate the report's "minimum ΔV table" for a fixed date, while still capturing strong sensitivity of ΔV_T to TOF in poorly aligned configurations.

4.2.4 Conversion from Cartesian State to Keplerian Elements

The function `cartesian_to_elements(r, v, mu)` converts a heliocentric state $(\mathbf{r}, \mathbf{v})$ to classical Keplerian elements following standard two-body relations [3].

Angular momentum and inclination. The specific angular momentum is

$$\mathbf{h} = \mathbf{r} \times \mathbf{v}, \qquad h = \|\mathbf{h}\|, \tag{74}$$

and the inclination is computed from the z-component:

$$i = \arccos\left(\frac{h_z}{h}\right). \tag{75}$$

Node vector and right ascension of the ascending node. With $\mathbf{k} = [0,0,1]^T$, the node vector is

$$\mathbf{n} = \mathbf{k} \times \mathbf{h}, \qquad n = \|\mathbf{n}\|. \tag{76}$$

If n is non-zero (non-equatorial orbit), the code obtains the longitude of ascending node via a quadrant-safe atan2:

$$\Omega = \text{atan2}(n_y, n_x). \tag{77}$$

If $n \approx 0$, the orbit is treated as equatorial and Ω is set to zero by convention.

Eccentricity vector and eccentricity magnitude. The eccentricity vector is computed as

$$\mathbf{e} = \frac{1}{\mu}\left[\left(v^2 - \frac{\mu}{r}\right)\mathbf{r} - (\mathbf{r}\cdot\mathbf{v})\mathbf{v}\right], \qquad e = \|\mathbf{e}\|. \tag{78}$$

Argument of periapsis. For non-equatorial orbits ($n > 0$) and non-circular cases ($e > 0$), the argument of periapsis is found from the angle between $\mathbf{n}$ and $\mathbf{e}$, with a quadrant fix using the sign of e_z. In the equatorial limit, ω is measured relative to the inertial x-axis using $\text{atan2}(e_y, e_x)$. For near-circular cases, ω is set to zero, consistent with the fact that periapsis is undefined when $e \approx 0$ [3].

Semi-major axis from specific orbital energy. The specific orbital energy is

$$\varepsilon = \frac{v^2}{2} - \frac{\mu}{r}, \tag{79}$$

and the semi-major axis follows as

$$a = -\frac{\mu}{2\varepsilon}, \tag{80}$$

with $a \to \infty$ in the parabolic limit $\varepsilon \to 0$.

4.2.5 Program Flow and Generated Output

The `main()` routine fixes a launch date and iterates over a list of target planets. For each target, it calls `find_min_dv` and prints one table row: ΔV_T (converted to km/s), the minimizing TOF, and the transfer-orbit elements computed from $(\mathbf{r}_E(t_{\text{dep}}), \mathbf{v}_{\text{tr},1})$. This directly supports the report requirement of producing a "minimum ΔV and orbital elements" table for a specified launch date.

Known limitations. The current implementation (i) searches only the prograde direction flag (`tm=+1`), (ii) does not explicitly scan long-way vs. short-way branches unless they emerge through the TOF grid and solver behavior, and (iii) does not include multi-revolution solutions. These modeling/algorithmic choices can shift the reported minimum away from the global optimum for certain geometries, which is consistent with the well-known multiplicity of Lambert solutions [9], [10].

4.3 Results

Table 6 summarizes the minimum heliocentric two-impulse cost ΔV_T and the associated transfer-orbit elements for a fixed launch date of 5 January 2026. The search procedure of Section 4 identified, for each planet, the time of flight (TOF) that minimizes ΔV_T within the scanned window. The values reflect the instantaneous geometry of the Solar System on that date and therefore are not globally optimal launch opportunities. *Note:* In the following table, "planets" refers to the eight IAU-defined planets (Mercury–Neptune). Pluto is treated separately as a dwarf planet and is therefore not included in this fixed-date planetary summary.

Table 6. Minimum ΔV and orbital elements for launch on 05/01/2026.

Planet	Δv **(km/s)**	**TOF (days)**	a **(AU)**	e	i (°)	Ω (°)	ω (°)
Mercury	23.313	86.6	0.7397	0.3766	4.45	284.24	340.28
Venus	12.058	269.9	0.9672	0.2485	3.56	104.34	251.88
Mars	28.004	197.0	0.9311	0.5970	0.09	106.41	128.73
Jupiter	19.645	2525.2	4.1530	0.7638	1.50	104.41	353.98
Saturn	36.137	6069.2	7.4011	0.9239	2.61	104.36	276.23
Uranus	23.563	16109.5	14.1714	0.9381	0.17	283.15	142.13
Neptune	39.371	30838.5	21.8212	0.9829	1.46	104.41	255.33

4.3.1 Interpretation of the Inner-Planet Transfers

Mars. On the chosen date, Mars lies at a heliocentric longitude of roughly 182° relative to Earth—almost exactly at superior conjunction (opposite the Sun). In this configuration, a standard minimum-energy Hohmann transfer is geometrically impossible, as the target planet is behind the Sun during Earth's departure window. Consequently, the optimizer identified a high-energy, eccentric trajectory ($e = 0.60$) requiring $\Delta V_T = 28.0$ km/s, nearly three times larger than that of a typical optimal Earth–Mars window (5–6 km/s round-trip equivalent [2], subsection 3.3). The semi-major axis $a = 0.93$ AU indicates that the spacecraft departs onto an ellipse that dips slightly inside Earth's orbit to gain energy before intersecting Mars's path near aphelion.

Venus. Venus exhibits a similar longitudinal opposition ($\sim 179°$), yet the required ΔV_T is more moderate (12.1 km/s). Because transfers toward the inner Solar System involve smaller changes in heliocentric energy, the cost penalty of poor alignment is less severe than for Mars. The computed eccentricity ($e = 0.25$) and semi-major axis ($a \approx 0.97$ AU) describe a low-energy ellipse typical of near-optimal Venus encounters even when not in a Hohmann phase.

Mercury. The Earth–Mercury result ($\Delta V_T = 23.3$ km/s, $a = 0.74$ AU, $e = 0.38$) reflects the large energy change needed to move the spacecraft's perihelion toward 0.39 AU. The narrow set of low-ΔV opportunities observed in the corresponding porkchop plots arises from Mercury's short period and high orbital speed, which make transfer geometry highly sensitive to departure epoch [3].

4.3.2 Outer-Planet Transfers and Scaling Trends

The ΔV_T values for Jupiter, Saturn, Uranus, and Neptune follow the expected scaling of heliocentric energy and phasing constraints: as target distance increases, both the time of flight and the characteristic energy rise approximately with orbital radius to the power of ½ [2]. Jupiter's case ($\Delta V_T = 19.6$ km/s, TOF 7 yr) is consistent with a typical direct transfer arc reaching ~ 5 AU. Beyond that, the optimizer finds only exceedingly long, multi-year ballistic trajectories: Saturn at 17 yr, Uranus at 44 yr, and Neptune at 85 yr. Their corresponding semi-major axes (4–22 AU) and eccentricities ($e = 0.76$–0.98) illustrate the increasingly elongated nature of direct Sun-centered solutions at outer distances. These durations and energy levels agree with textbook estimates for single-impulse patched-conic rendezvous and underline why realistic outer-planet missions rely on gravity assists (e.g., Voyager, Galileo) to lower ΔV [14].

4.3.3 Orbital-Element Patterns

Across all planets, the inclination values remain below 5°, consistent with the nearly coplanar ecliptic. The clustering of Ω around 100°–110° corresponds to the ecliptic orientation of Earth's orbit relative to the inertial reference frame (J2000). The spread in ω—from 128° to 355°—captures the wide variation in transfer-ellipse orientation induced by each planet's instantaneous phase.

4.3.4 Summary

The computed ΔV_T levels are physically consistent with the actual planetary geometry of early 2026. Inner-planet transfers exhibit modest sensitivity but high costs under opposition conditions, while outer-planet targets demand multi-year, high-eccentricity trajectories. These results illustrate the central mission-analysis principle that transfer feasibility and energy are dominated by orbital phasing rather than by distance alone [2], [3], [9].

5 COMPUTATION TIME

Timing setup and assumptions. A single pork-chop plot (PCP) run is defined here as computing ΔV_T over a 2D grid of $N_{\text{dep}} \times N_{\text{arr}}$ departure/arrival epochs for one target planet and one fixed time window, using the same Lambert-solver settings (branch selection and iteration tolerances) throughout. The reported times were measured on `Intel i5 10300H` and `Intel i7 4710MQ` at two different resolution levels and include the full grid evaluation.

The computation time for generating planetary transfer pork-chop plots is highly dependent on the resolution chosen for the mesh. As expected, the computational cost increases significantly with higher resolution due to the square scaling of the mesh size with resolution. Specifically, the computational cost is expected to increase quadratically with the resolution, since this variable represents the number of divisions in each axis.

Table 7 summarizes the computation times for two different CPUs, the Intel i5 10300H and Intel i7 4710MQ, at two different resolution levels (500 and 50).

Table 7. Computation Times for Different CPUs and Resolutions

CPU	Resolution	Computation Time
Intel i5 10300H	500 x 500	160.45 seconds
Intel i5 10300H	50 x 50	1.60 seconds
Intel i7 4710MQ	500 x 500	1067.46 seconds
Intel i7 4710MQ	50 x 50	10.77 seconds

These results demonstrate the significant effect of resolution on the computation time and the efficiency of running these programs in fast CPUs, as the computational costs can be very demanding.

6 CONCLUSION

Summary

This report developed and verified a Python-based interplanetary mission-analysis pipeline for preliminary launch-window exploration in a Sun-centered two-body (patched-conic) framework. The toolchain couples (i) analytical planetary ephemerides, (ii) a Lambert solver based on the Simo formulation with a multi-revolution extension, and (iii) application routines for pork-chop plot generation and fixed-date minimum-ΔV targeting.

Validation against JPL Horizons confirmed correct ephemerides implementation and the expected accuracy limits of a mean-element secular model: discrepancies are small for inner-planet states but increase for outer targets. The Lambert solver reproduced canonical benchmark cases and a reference GUI, achieving time-of-flight residuals close to numerical precision across elliptic/hyperbolic regimes and for different transfer branches; the multi-revolution extension recovered the expected multiple-solution structure. Using these validated components, pork-chop plots successfully revealed low-cost basins and strong sensitivity regions in $(t_{\mathrm{dep}}, \mathrm{TOF})$ space. For the representative fixed launch date 05/01/2026, the minimum two-impulse costs within the scanned domain were lowest for Venus ($\Delta V_T \approx 12.06$ km/s, TOF ≈ 270 d), while unfavorable phasing produced substantially higher costs for several targets (e.g., Mars $\Delta V_T \approx 28.00$ km/s). Direct ballistic outer-planet transfers required very long times of flight and high-eccentricity trajectories, highlighting the need for gravity assists and/or multi-leg strategies for realistic distant missions.

Finally, the runtime study confirmed an approximately quadratic scaling with PCP grid resolution, making resolution a key design parameter for practical surveys. Overall, the framework provides a transparent baseline for early-phase trade studies; future work should focus on higher-fidelity ephemerides, systematic scanning of all Lambert branches (short/long-way, both directions, multi-rev), and more efficient search strategies (adaptive refinement and parallelization), with gravity-assist sequencing as the natural next step for outer-planet missions.

Learnings and difficulties

Key learnings.

- **Model hierarchy matters:** fast analytical ephemerides are highly effective for screening and visualization, but their accuracy limitations become more evident for outer planets and long horizons, motivating clear separation between “trade study” and “high-fidelity” use cases.
- **Lambert solutions are not unique:** short/long-way transfers, direction choices, and multi-revolution families can produce multiple valid solutions for the same boundary conditions; robust tooling must therefore treat “branch management” as a first-class concern.
- **Phasing dominates:** the fixed-date analysis highlighted that transfer cost does not scale monotonically with distance; unfavorable geometry can yield large ΔV even for nearer targets.
- **Verification is essential:** cross-checking ephemerides against a trusted reference and testing Lambert solutions against benchmark cases/GUI is necessary to trust subsequent PCP minima and orbital-element extraction.

Main difficulties.

- **Numerical robustness near edge cases:** convergence and stability become more delicate for near-parabolic trajectories, extreme times of flight, and multi-revolution configurations, requiring careful initial guesses, tolerances, and stopping criteria.
- **Consistent units and frames:** maintaining strict consistency between AU/day internal units and reported km/s, and ensuring correct frame conventions (ecliptic/heliocentric) is crucial; small inconsistencies can produce large apparent errors in ΔV.

- **Search vs. runtime trade-off:** brute-force PCP scanning provides transparency and reliability but scales strongly with grid resolution; runtime quickly becomes the limiting factor when aiming for fine resolution across wide launch windows.

Future improvements

The current framework provides a validated baseline for early-phase surveys. The most impactful extensions to improve accuracy, completeness, and efficiency are:

- **Higher-fidelity ephemerides:** replace/augment the analytical model with numerical ephemerides (e.g., SPICE/JPL kernels) for improved absolute accuracy, particularly for outer planets and long time spans.
- **Systematic branch scanning:** ensure that PCP and fixed-date searches consistently explore short/long-way solutions, both directions, and multi-revolution families to reduce the risk of missing the global minimum.
- **Broken Plane Maneuver (BPM):** implement the logic for mid-course plane changes to optimize transfers between high-inclination orbits, avoiding the inefficiencies of forcing a single-plane Lambert solution.
- **More efficient window search:** introduce adaptive mesh refinement around low-ΔV valleys, parallelize Lambert calls, and/or use gradient-free optimizers to reduce the computational cost of high-resolution studies.
- **More realistic mission metrics:** complement ΔV_T with launch energy (e.g., C_3), departure/arrival v_∞ constraints, and operational constraints (maximum TOF, bounded arrival conditions).
- **Beyond single-leg ballistic transfers:** add gravity assists and multi-leg patched-conic sequencing for realistic outer-planet mission design, where direct transfers are typically not competitive.
- **CLI revision and error handling:** address known runtime exceptions within the Command Line Interface to improve code stability, input validation, and user feedback during execution.

REFERENCES

[1] E. M. Standish and J. G. Williams, "Orbital ephemerides of the sun, moon, and planets," in *Explanatory Supplement to the Astronomical Almanac*, S. E. Urban and P. K. Seidelmann, Eds., 3rd ed., Mill Valley, CA: University Science Books, 2013, pp. 305–345.

[2] H. D. Curtis, *Orbital Mechanics for Engineering Students*, 4th ed. Butterworth-Heinemann, 2020, ISBN: 9780128240250.

[3] D. A. Vallado, *Fundamentals of Astrodynamics and Applications* (Space Technology Library), 4th ed. Hawthorne, CA: Microcosm Press, 2013.

[4] International Astronomical Union, "Iau 1976 system of astronomical constants," IAU, Tech. Rep., 1976, Adopted system of astronomical constants, including the Gaussian gravitational constant.

[5] International Astronomical Union, *Resolution b2 on the re-definition of the astronomical unit of length*, XXVIIIth General Assembly of the IAU, Beijing, https://syrte.obspm.fr/IAU_resolutions/Res_IAU2012_B2.pdf, 2012.

[6] J. D. Giorgini and JPL Solar System Dynamics Group, *The JPL horizons on-line ephemeris system*, https://ssd.jpl.nasa.gov/horizons/, Accessed: 2025-12-02.

[7] A. Ginsburg et al., "Astroquery: An astronomical web-querying package in python," *The Astronomical Journal*, vol. 157, no. 3, p. 98, 2019. DOI: 10.3847/1538-3881/aafc33.

[8] G. De La Torre, E. Fantino, and R. Flores, "A graphical user interface for solving lambert's problem," *Acta Astronautica*, vol. 153, pp. 26–38, 2018, Open PDF available via SciPedia (license shown on the page). DOI: 10.1016/j.actaastro.2018.04.050. [Online]. Available: https://www.scipedia.com/public/De_La_Torre_et_al_2018a.

[9] D. Izzo, "Revisiting lambert's problem," *Celestial Mechanics and Dynamical Astronomy*, vol. 121, no. 1, pp. 1–15, 2015. DOI: 10.1007/s10569-014-9587-y. arXiv: 1403.2702 [math.OC].

[10] J. R. Gatland, "A review of lambert's problem," 2021. arXiv: 2105.06737 [astro-ph.EP]. [Online]. Available: https://arxiv.org/abs/2105.06737.

[11] NASA Ames Research Center, *Trajectory browser user guide*, https://trajbrowser.arc.nasa.gov/user_guide.php, Accessed: 2026-01-05.

[12] E. R. Lancaster and R. C. Blanchard, "A unified form of lambert's theorem," National Aeronautics and Space Administration (NASA), Goddard Space Flight Center, Tech. Rep. NASA TN D-5368, 1969. [Online]. Available: https://ntrs.nasa.gov/api/citations/19690027552/downloads/19690027552.pdf.

[13] JPL Solar System Dynamics Group, *Horizons system manual*, https://ssd.jpl.nasa.gov/horizons/manual.html, Version 4.98d (2025-11-21), Accessed: 2026-01-05.

[14] J. M. Longuski, J. J. Guzmán, and J. E. Prussing, *Optimal Control with Aerospace Applications*. Springer, 2014. DOI: 10.1007/978-1-4471-5626-3.